CPTS 489 Web Application Development

Final Project Report

Home Food Marketplace

Team Members: Yaru Gao

Semester: Spring 2026

Date: 2026-04-20

1. Introduction and Project Overview

Home Food Marketplace is a community-focused web application that connects local home cooks with nearby customers who want affordable home-cooked meals. The system supports two active roles: customers and merchants (cooks). Customers can register, log in, browse restaurants, filter by tags, manage cart and checkout, and track order status. Merchants can create and manage restaurants, assign cuisine tags, maintain menu inventory, and handle incoming orders through a dashboard. The application is built with Express, Sequelize, EJS, and MySQL, with session-based authentication and role-based access control for protected routes.

2. Implementation Documentation (Required)

2.1 Section 1: MVC Implementation Walkthrough

This section covers all use cases through an MVC walkthrough, showing how each one is implemented across routes, controllers, models, and views.

Use Case 1 (UC-G01): User Login

1. The Route

Files: HFM/routes/auth.js

When a user opens /login, the server returns the login form view so credentials can be entered. After the form is submitted, a POST request is sent to /login where the route validates required fields and forwards the request to authentication logic. If credentials are valid, the route initializes session state (userId and role) and redirects the user to the role-appropriate dashboard; otherwise, it re-renders the login page with an error message.

2. The Controller

File: HFM/routes/auth.js | Function: router.post('/login')

The controller is responsible for all business logic. It executes the following steps:

1. Reads email and password from req.body and validates both fields are present.
2. Calls User.findUser(email, password) to fetch the account and verify bcrypt password.
3. If authentication fails, re-renders login with an invalid-credentials error.
4. If authentication succeeds, writes userId and role into session state.
5. Redirects cook users to /merchant-dashboard and customer users to /customer-dashboard.

3. The Models

This use case reads from and writes to the following model(s):

Model	Operation	What Happens

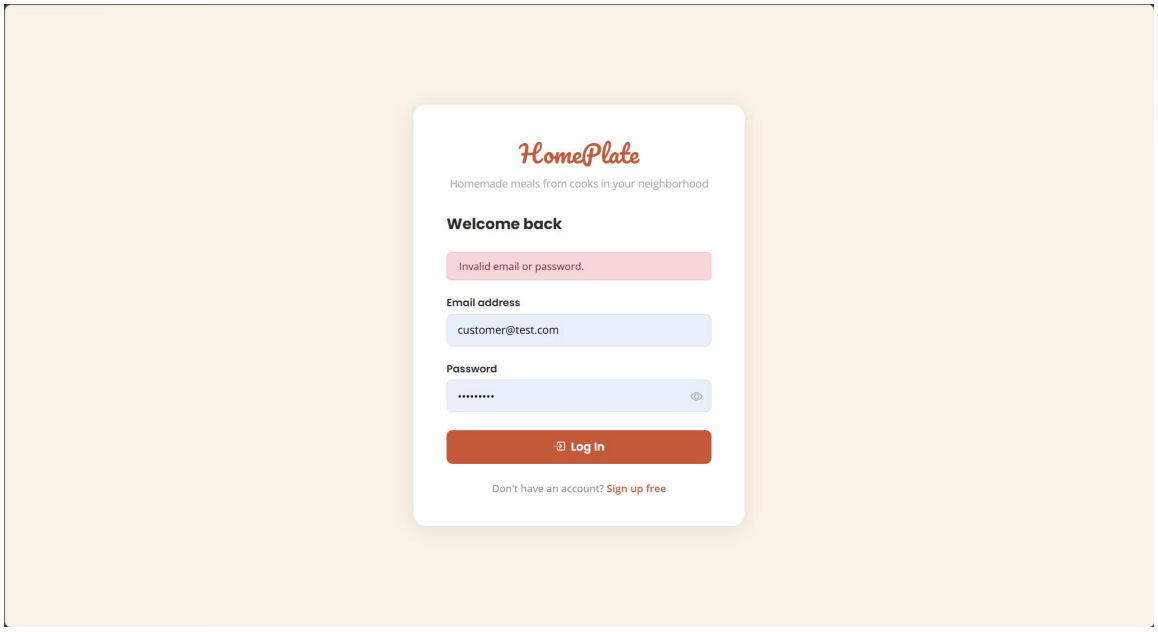
User	Read	Looks up the account by email and verifies hashed password.

4. The Views

This use case involves the following view(s):

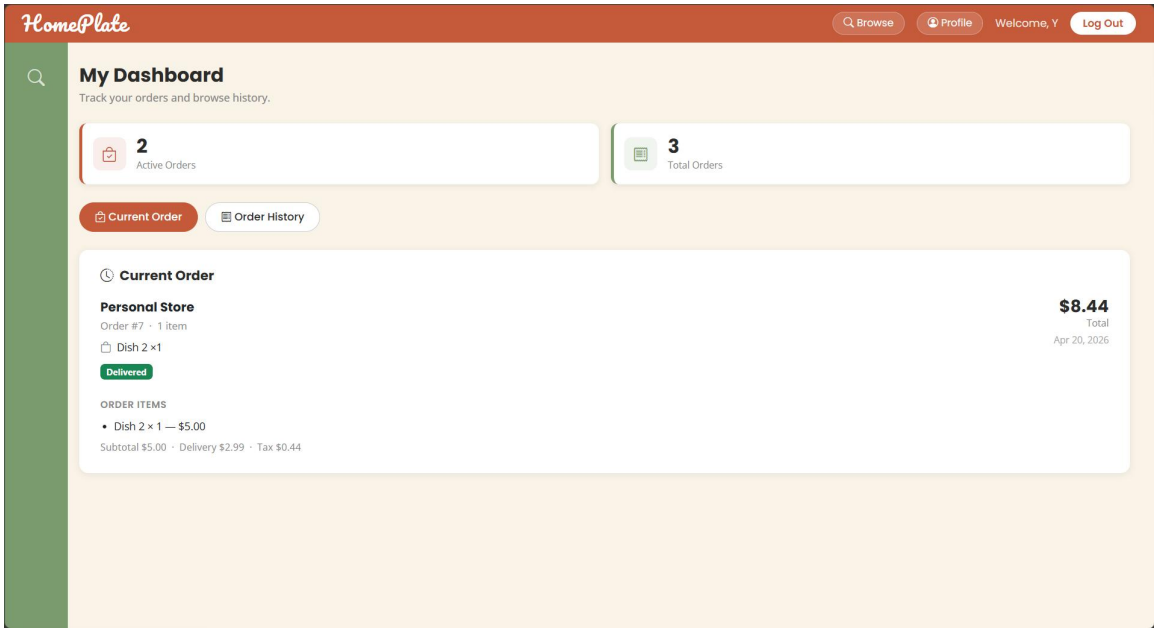
View -- Login Form (file: HFM/views/login.ejs)

This is the page where the user enters email and password. Submitting this form triggers POST /login and starts authentication for UC-G01. The form includes fields for email, password, and visual indication of login status.



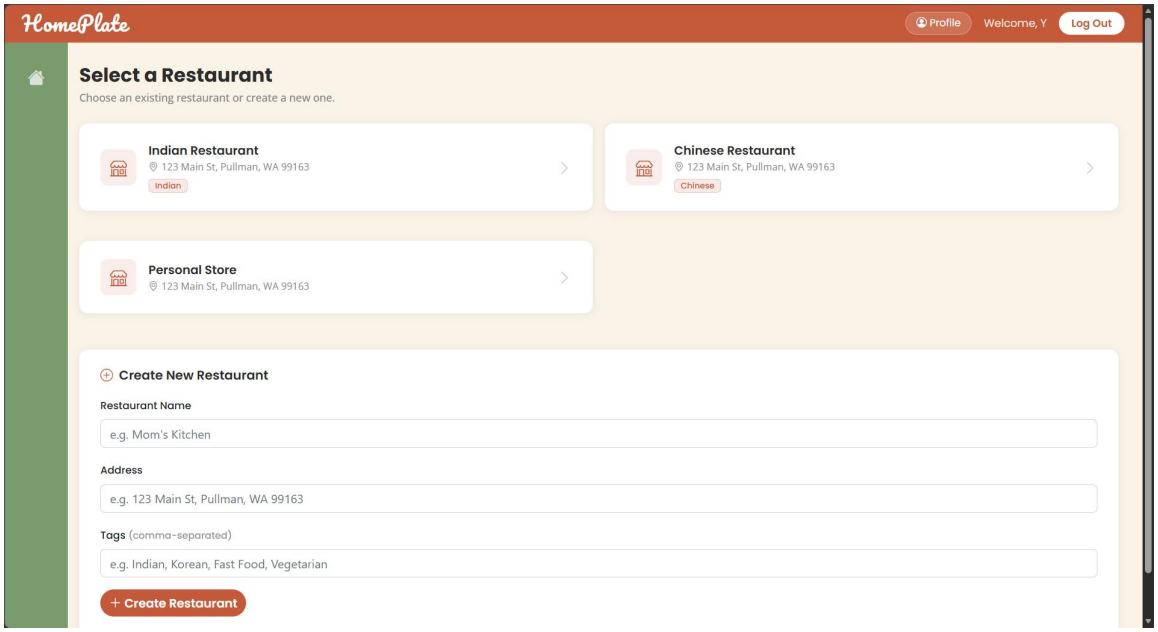
View -- Customer Dashboard After Login (file: HFM/views/customer-dashboard.ejs)

When a customer logs in, they are redirected to their role-specific dashboard. This view confirms login success, displays role-specific navigation, and provides entry points to browse, cart, and order history features.



View -- Merchant Dashboard After Login (file: HFM/views/merchant-dashboard.ejs)

When a merchant (cook) logs in, they are redirected to the merchant dashboard. This view displays restaurant selection interface and provides access to menu management, order fulfillment, and reporting features.



Use Case 2 (UC-G02): User Registration (Sign Up)

1. The Route

Files: HFM/routes/auth.js

The registration flow starts at GET /register, which serves the sign-up form for new users. Submitting the form sends POST /register with identity, credential, and role inputs. The route performs validation (required fields, password rules, confirmation match), selects the correct model for the chosen role, and creates the account. On success, the user is redirected to /login; on failure, the same register view is returned with a clear validation or duplicate-email message.

2. The Controller

File: HFM/routes/auth.js | Function: router.post('/register')

The controller is responsible for all business logic. It executes the following steps:

1. Reads registration payload (name, email, password, confirmPassword, role) from req.body.
2. Validates required fields, minimum password length, and password confirmation match.
3. Normalizes role so only 'customer' or 'cook' is accepted (fallback to customer).
4. Selects Merchant or Customer model based on role and creates the account record.
5. Handles duplicate-email constraint errors and returns a user-facing registration message.
6. On success, redirects to /login for first authentication.

3. The Models

This use case reads from and writes to the following model(s):

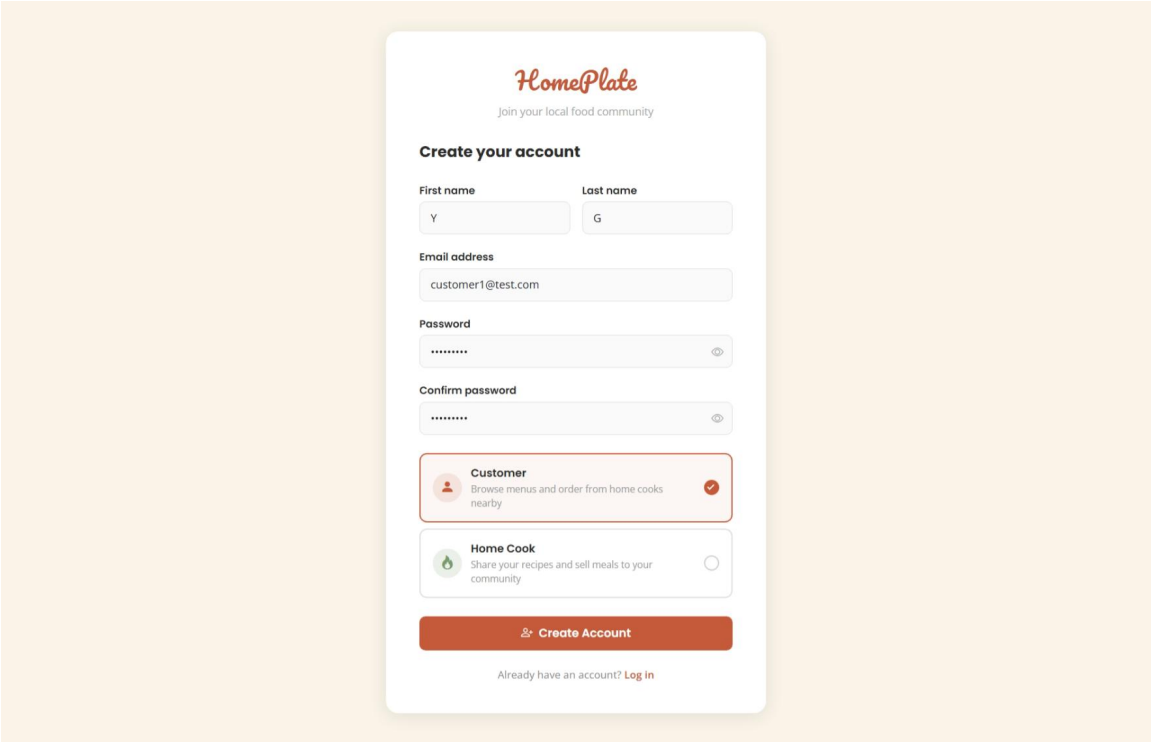
Model	Operation	What Happens
Customer	Create	Creates a customer record with role and hashed password when role=customer.
Merchant	Create	Creates a merchant record with role and hashed password when role=cook.

4. The Views

This use case involves the following view(s):

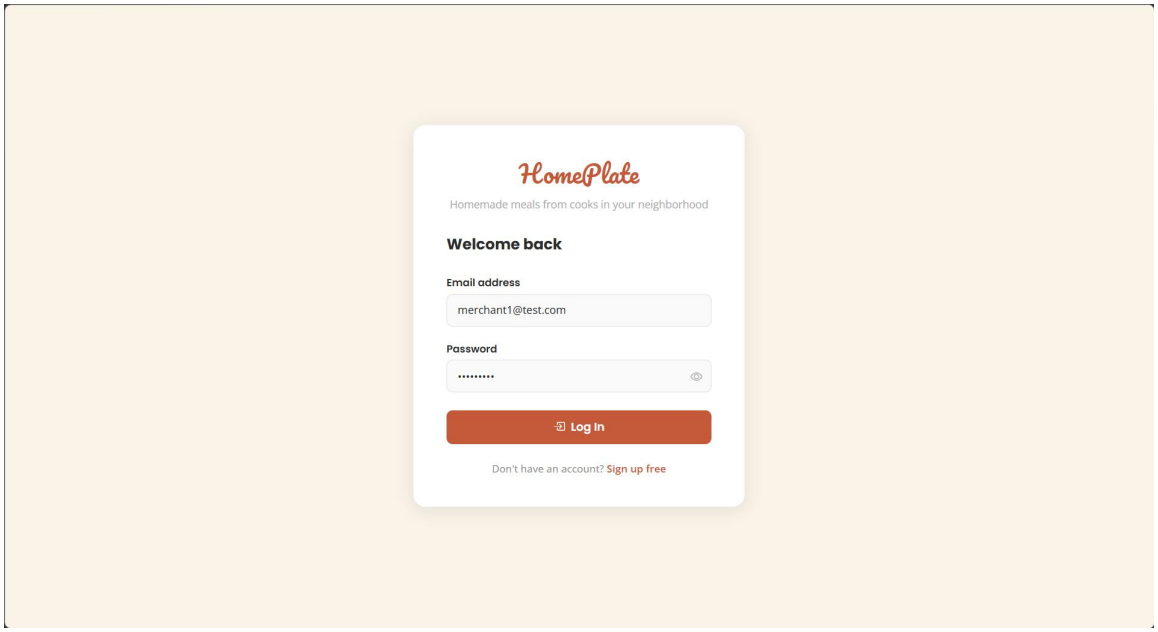
View -- Registration Form (file: HFM/views/register.ejs)

This page captures user identity (first name, last name, email), password (with confirmation), and role selection. The page displays role selection options (Customer or Cook/Merchant) allowing users to define their account type before creation. Submitting the form triggers POST /register and account creation for UC-G02.



View -- Login Page After Registration Redirect (file: HFM/views/login.ejs)

After successful registration, the user is redirected to the login page to authenticate with their newly created credentials. This view confirms account creation was successful and transitions to the UC-G01 authentication flow. The page may display a success message or transition context if applicable.



Use Case 3 (UC-C01): Browse and Search Food Listings

1. The Route

Files: HFM/routes/browse.js, HFM/routes/restaurant-detail.js

Customers enter this flow through GET /browse, which reads optional query parameters (keyword and tag) and prepares a filtered restaurant list for rendering. The route also builds supporting browse metadata such as available tags and card-level display fields. When a restaurant is selected, the browser navigates to GET /restaurant-detail?id=<id>, where the server loads the restaurant profile, owner context, and available menu items grouped by category before returning the detail page.

2. The Controller

File: HFM/routes/browse.js, HFM/routes/restaurant-detail.js | Function: router.get('/browse'), router.get('/restaurant-detail')

The controller is responsible for all business logic. It executes the following steps:

1. Loads optional session user context for personalized navigation state.
2. Builds browse query conditions from q and loads restaurants in reverse creation order.
3. Extracts and normalizes all available tags from restaurant records for filter UI.
4. Applies tag filter to the result set when tag parameter is provided.
5. For each listed restaurant, computes menu count and owner name for card rendering.
6. On detail route, loads selected restaurant, owner profile, and available menu items.
7. Groups menu items by category and computes cart badge count for the active customer session.

3. The Models

This use case reads from and writes to the following model(s):

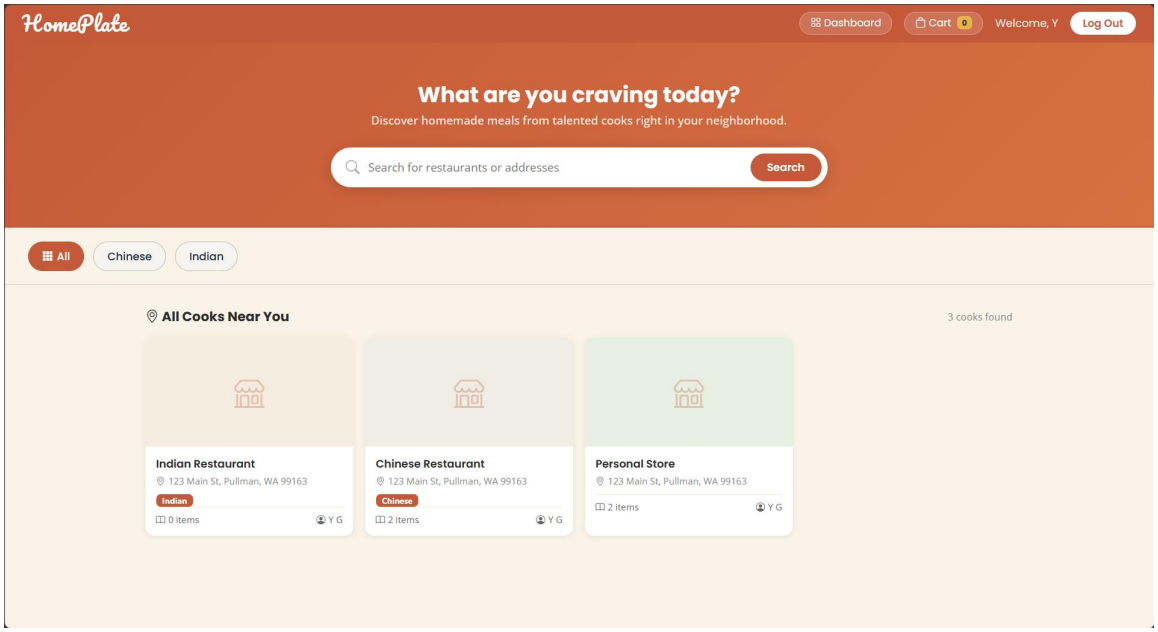
Model	Operation	What Happens
Restaurant	Read	Returns restaurant list and detail record for browse and selection.
MenuItem	Read	Counts/lists available items for cards and detail categories.
User	Read	Resolves owner name and current session user context.
Merchant	Read	Loads merchant-specific profile fields for detail page owner block.

4. The Views

This use case involves the following view(s):

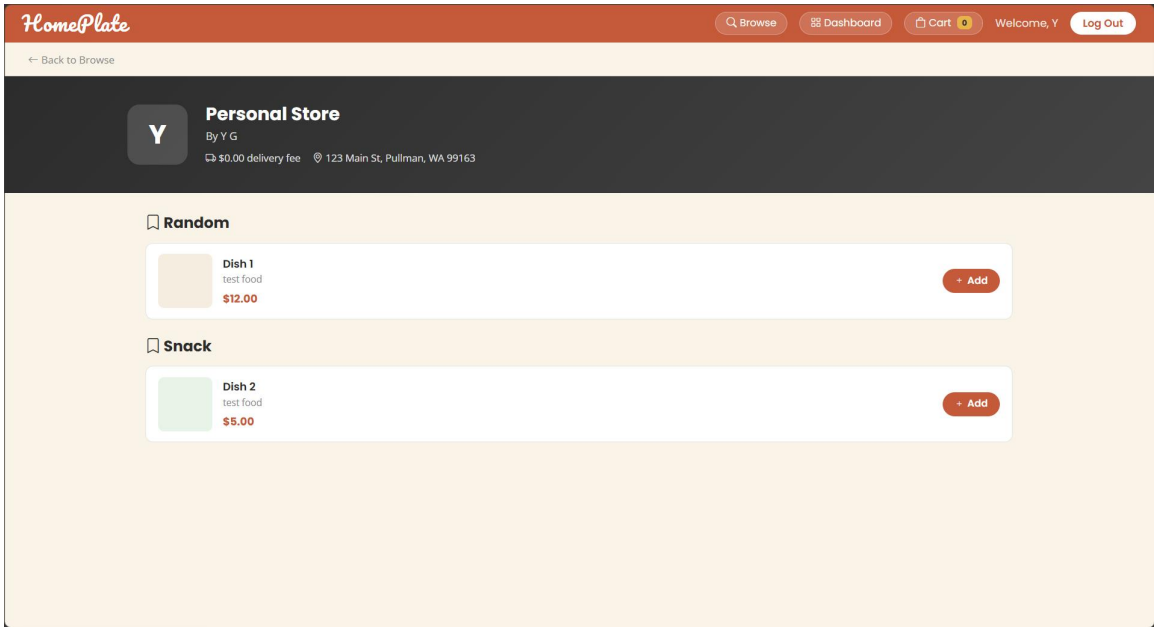
View -- Browse Restaurants Page (file: HFM/views/browse.ejs)

This page shows searchable restaurant cards with tag filters. Customers can enter search keywords or select available tags to filter results. Each restaurant card displays name, address, tags, and a thumbnail. Clicking a card navigates to the restaurant detail page. This is the entry action for UC-C01.



View -- Restaurant Detail with Menu Categories (file: HFM/views/restaurant-detail.ejs)

After selecting a restaurant, the detail page displays the restaurant name, address, owner information, and menu items grouped by category. Each category shows relevant menu items with name, price, description, and "Add to Cart" button. This is the completion view for discovering a specific listing.



Use Case 4 (UC-C02): Manage Cart

1. The Route

Files: HFM/routes/cart.js

Cart management combines one page route and multiple mutation endpoints. GET /cart loads all pending order groups for the logged-in customer and renders item-level totals. POST /cart/add handles add-to-cart actions by creating or reusing a pending order and then inserting or incrementing the corresponding line item. PUT /cart/item/:id updates quantities, and DELETE /cart/item/:id removes lines. After each mutation, totals are recalculated so the cart always reflects current subtotal, tax, delivery fee, and total.

2. The Controller

File: HFM/routes/cart.js | Function: router.post('/cart/add'), router.put('/cart/item/:id'), router.delete('/cart/item/:id')

The controller is responsible for all business logic. It executes the following steps:

1. Enforces login for all cart mutation endpoints via requireLogin middleware.
2. On add-item, validates menuItemId/restaurantId, loads menu item and restaurant, then finds or creates a pending order scoped by customer and restaurant.
3. Finds existing order line by orderId+menuItemId and either increments quantity or creates a new OrderItem snapshot.
4. Recalculates subtotal, tax, delivery fee, and total after each cart mutation via recalcOrder().
5. On quantity update, removes the line (and possibly the empty order) when quantity <= 0; otherwise saves the new quantity.
6. On delete, removes the line item, deletes empty pending orders, and returns updated cart counters.

3. The Models

This use case reads from and writes to the following model(s):

Model	Operation	What Happens
Order	Read/Create/Update/Delete	Maintains pending cart container and recalculated totals.
OrderItem	Read/Create/Update/Delete	Stores per-item quantities and removes items as needed.
MenuItem	Read	Validates item existence while adding to cart.

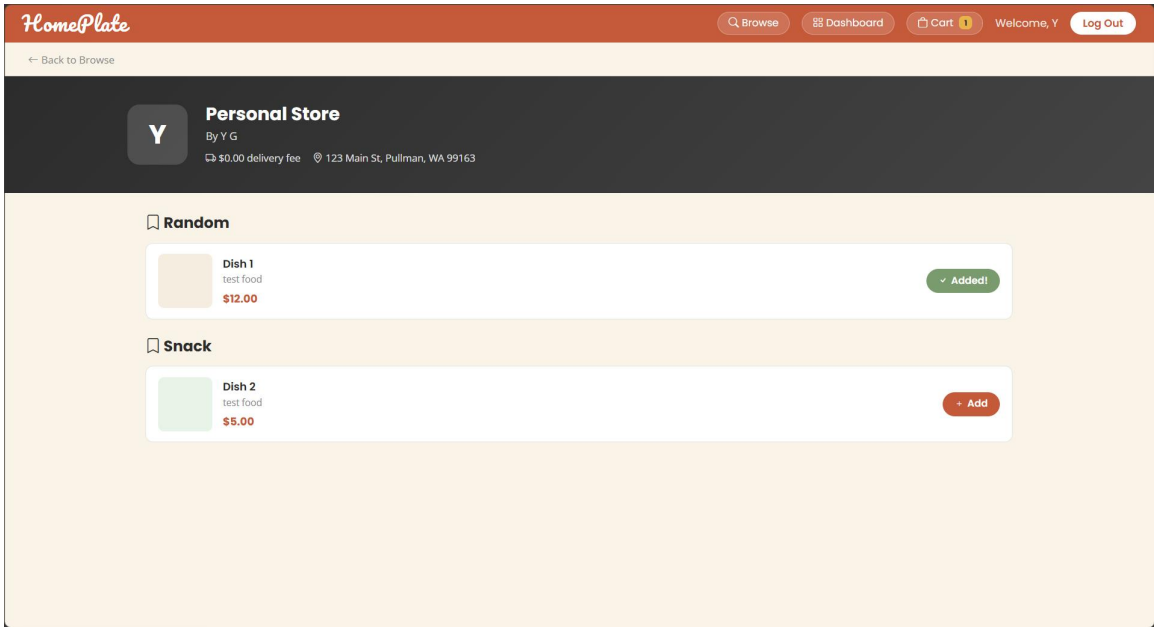
Restaurant	Read	Resolves restaurant ownership when creating pending order.

4. The Views

This use case involves the following view(s):

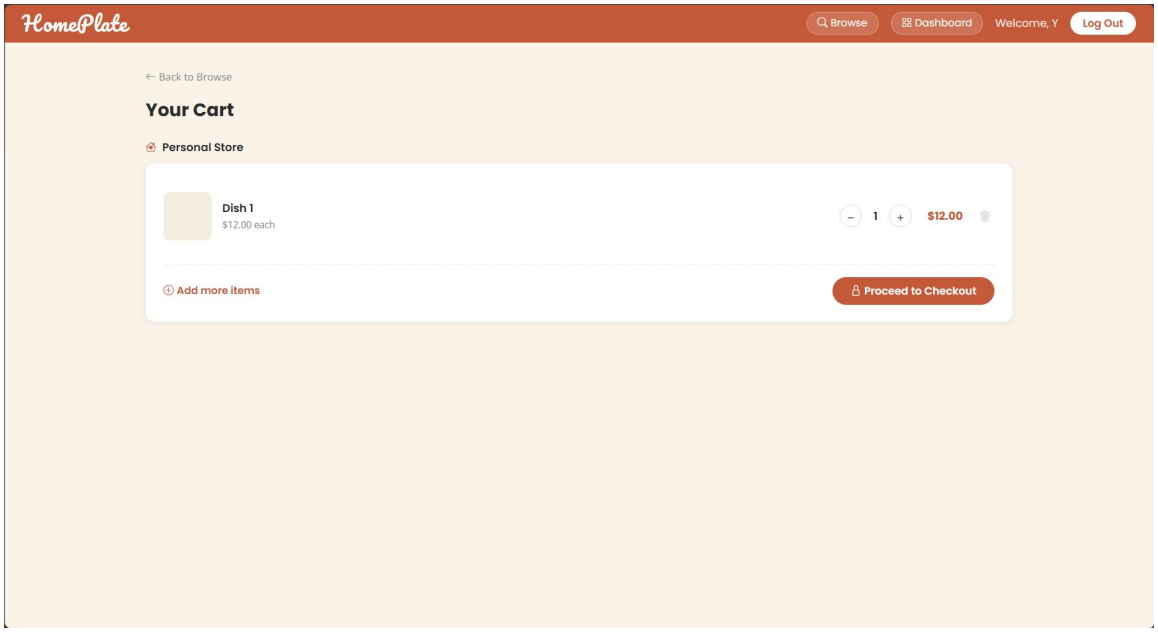
View -- Restaurant Detail with Add-to-Cart (file: HFM/views/restaurant-detail.ejs)

Customers add items to cart from the restaurant detail page by selecting items and clicking "Add to Cart". This initiates cart creation or quantity increment behavior in UC-C02. The page displays a cart badge showing current item count.



View -- Cart Management Page (file: HFM/views/cart.ejs)

This page shows pending orders grouped by restaurant. For each restaurant group, customers can view items with quantity controls (+/- buttons), remove individual items, and see recalculated totals (subtotal, tax, delivery fee, total). It is the primary management view for UC-C02 with navigation to checkout.



Use Case 5 (UC-C03): Checkout and Place Order

1. The Route

Files: HFM/routes/checkout.js

The checkout page is served by GET /checkout, which resolves the target pending order (either by explicit orderId or latest pending order), loads items, and renders the checkout form. When the customer places the order, POST /checkout validates session ownership and required routing inputs, then executes the placement logic in a transaction. The route verifies stock, writes delivery/fulfillment values, changes order status from pending to placed, and finally redirects to /order-confirmation with the new order id.

2. The Controller

File: HFM/routes/checkout.js | Function: router.get('/checkout'), router.post('/checkout')

The controller is responsible for all business logic. It executes the following steps:

1. GET handler validates session and resolves the target pending order by explicit orderId or latest pending order.
2. Loads restaurant and order items for page rendering, and redirects back to cart when order is missing or empty.
3. POST handler validates session, orderId, and normalized delivery state input.
4. Opens a database transaction and lock-checks the pending order ownership (customerId + status=pending).
5. Loads order items with lock, verifies non-empty cart, and validates each menu item still exists with sufficient stock.

6. Deducts inventory quantities, updates availability flags, and saves each menu item within the same transaction.
7. Persists delivery and fulfillment fields to the order, then updates status from pending to placed.
8. After commit, schedules simulated auto-delivery and redirects to order confirmation page.

3. The Models

This use case reads from and writes to the following model(s):

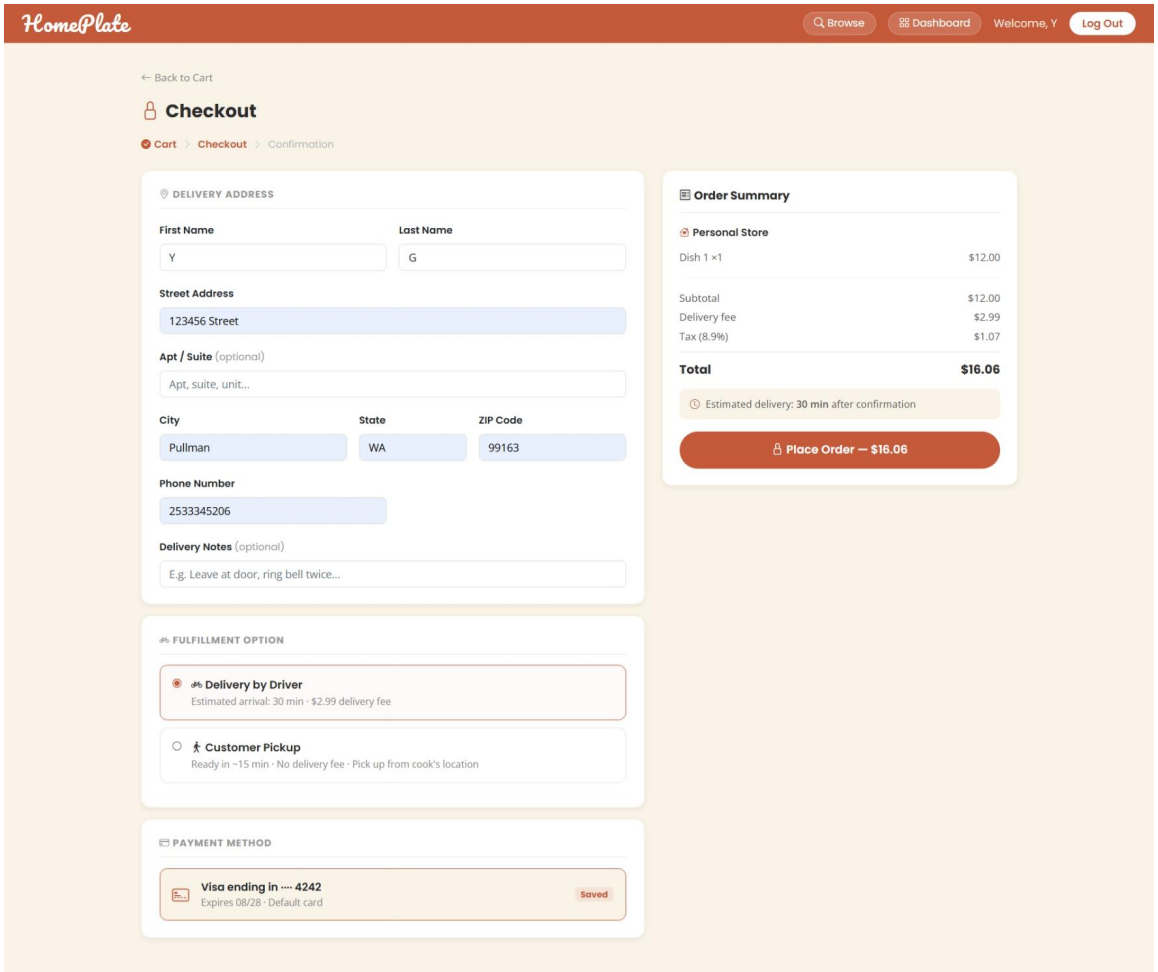
Model	Operation	What Happens
Order	Read/Update	Stores delivery details and updates status from pending to placed.
OrderItem	Read	Provides purchased item rows for stock checks and confirmation display.
MenuItem	Read/Update	Validates stock and decrements quantity/availability.
Restaurant	Read	Supplies restaurant context for checkout and confirmation page.
User	Read	Supplies customer context for page rendering.

4. The Views

This use case involves the following view(s):

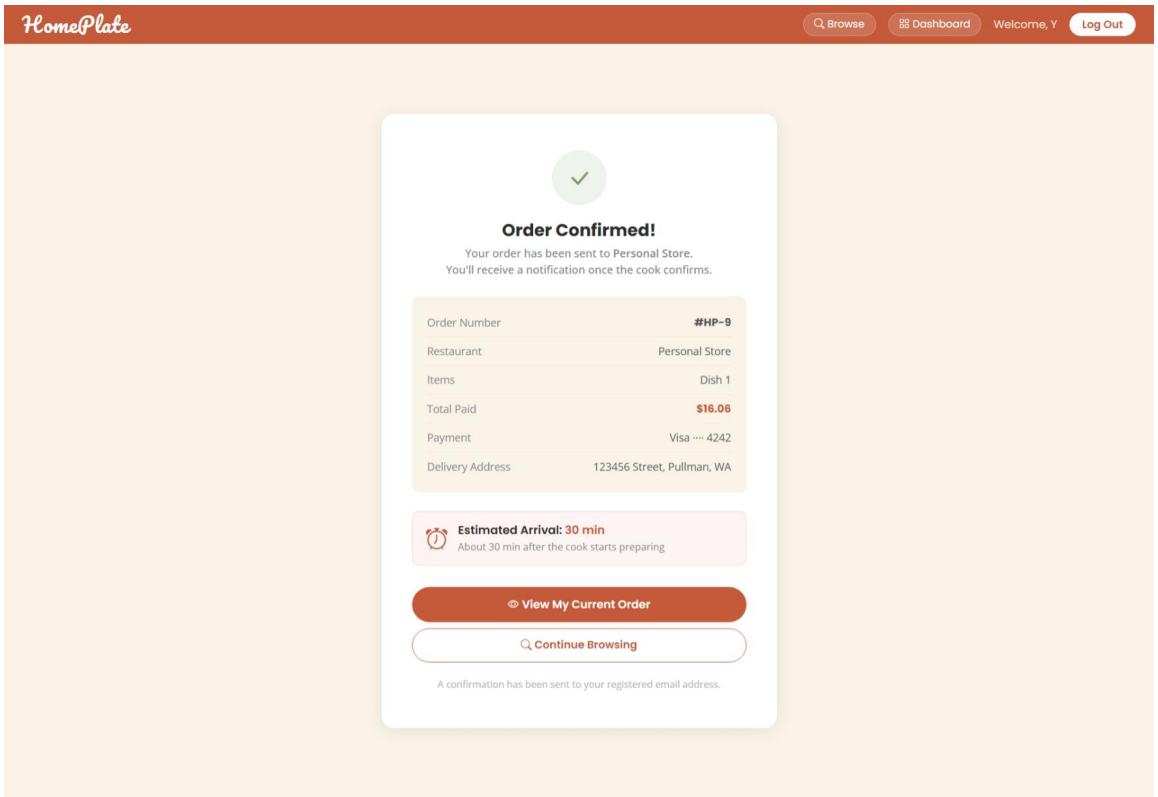
View -- Checkout Form (file: HFM/views/checkout.ejs)

This page displays the order summary (grouped items with quantities and prices), subtotal/tax/delivery fee/total, and delivery information form fields (street, city, state, zip, fulfillment method selector). The user reviews the order and enters delivery details before submitting. Submitting the form triggers POST /checkout action for UC-C03.



View -- Order Confirmation Page (file: HFM/views/order-confirmation.ejs)

After successful checkout, this confirmation page renders the newly created order with its ID, complete line items with quantities and prices, restaurant name, delivery address, and calculated totals (subtotal, tax, delivery fee, total). This is the final user-facing success view of UC-C03.



Use Case 6 (UC-C04): View Order History

1. The Route

Files: HFM/routes/customer.js

This use case is routed through customer-only endpoints protected by role checks. GET /customer-dashboard renders the dashboard shell and summary counts. GET /customer-current-order fetches and renders the most recent active (non-pending) order fragment, while GET /customer-order-history-data returns the historical order list fragment with item and restaurant context. Together, these routes provide both current-status visibility and full historical review for customer orders.

2. The Controller

File: HFM/routes/customer.js | Function: router.get('/customer-dashboard'), router.get('/customer-current-order'), router.get('/customer-order-history-data')

The controller is responsible for all business logic. It executes the following steps:

1. Applies requireCustomer middleware to enforce authenticated customer-role access.
2. Dashboard handler counts active orders (placed/confirmed/ready) and total historical orders (status != pending).
3. Current-order handler fetches the latest non-pending order for the customer.

4. Loads line items and restaurant context for that current order when present.
5. History handler fetches all non-pending orders in descending creation order.
6. For each historical order, attaches order items and restaurant details before rendering the fragment view.

3. The Models

This use case reads from and writes to the following model(s):

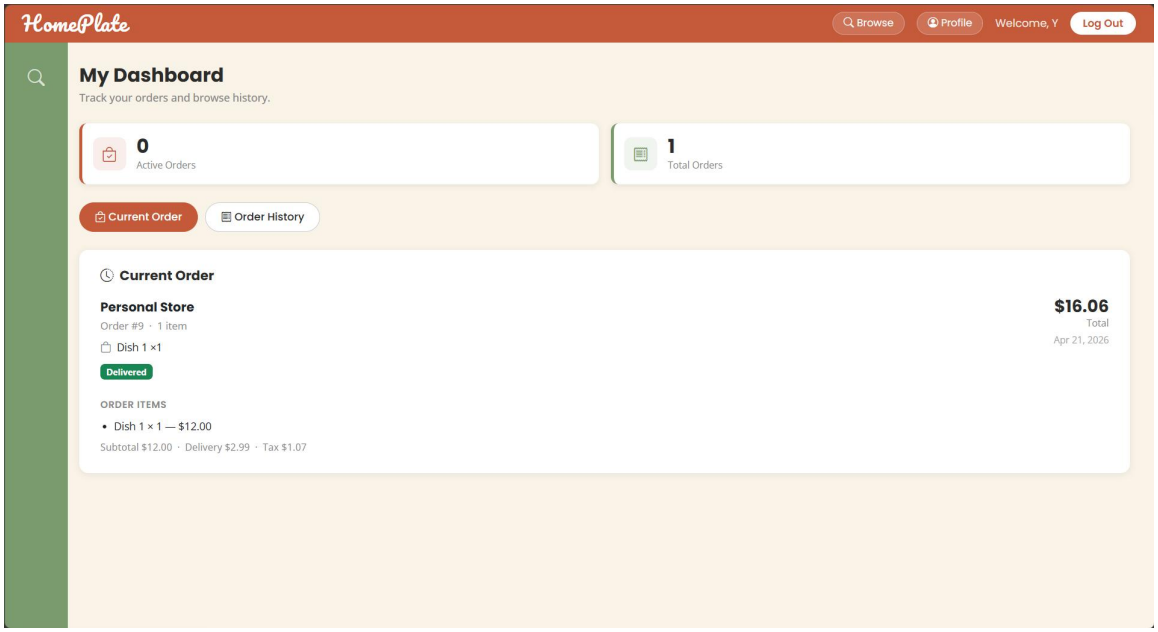
Model	Operation	What Happens
Order	Read	Retrieves active and historical customer orders by status.
OrderItem	Read	Retrieves line-item detail for each historical order.
Restaurant	Read	Attaches restaurant identity to each order card.
User	Read	Provides customer identity and dashboard context.

4. The Views

This use case involves the following view(s):

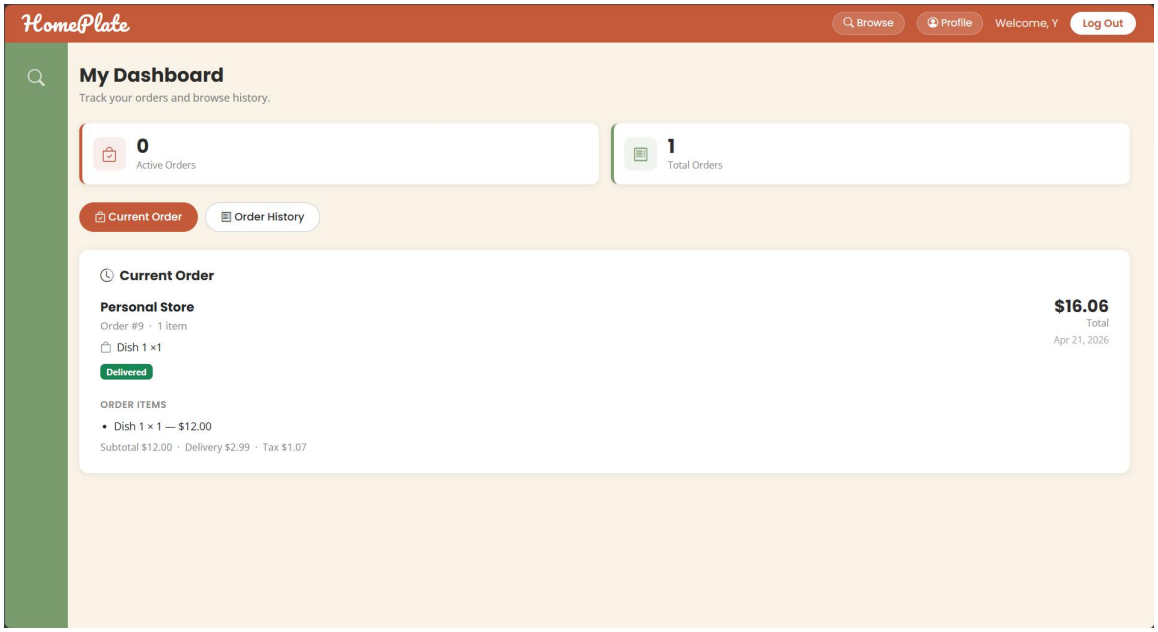
View -- Customer Dashboard Hub (file: HFM/views/customer-dashboard.ejs)

The dashboard hosts tabs for current order status and order history. This is the entry point where the customer chooses which order-related view to access. The dashboard displays stat cards showing counts of active orders and total completed orders.



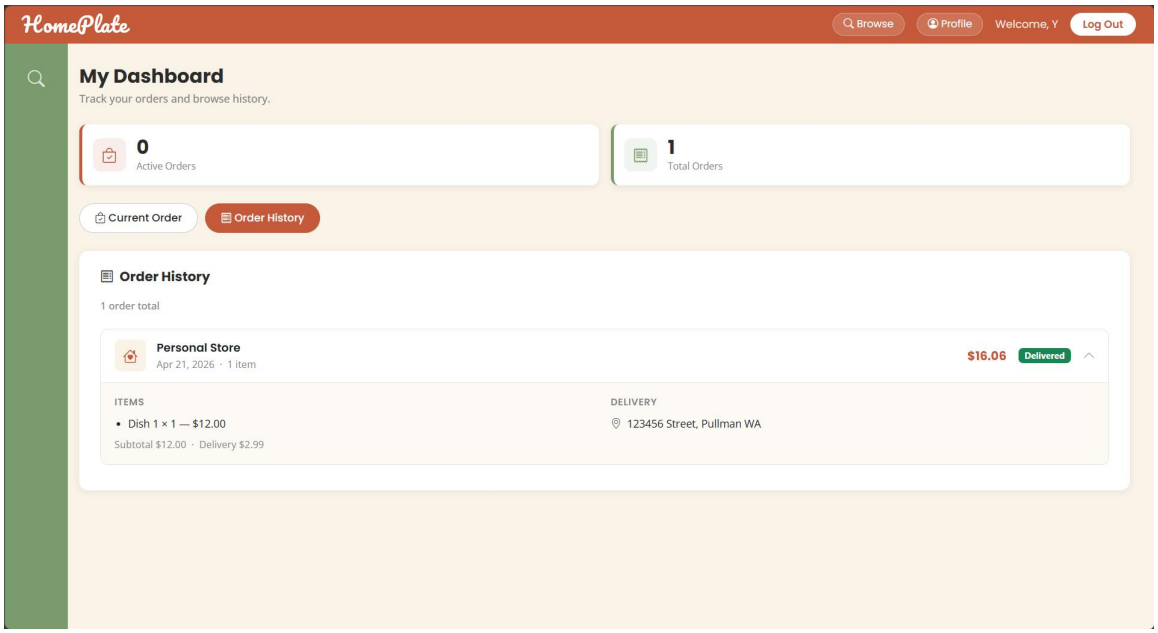
View -- Current Order Status View (file: HFM/views/customer-order-status.ejs)

This view displays the most recent non-pending order (current active order) with its status, restaurant name, order items, and timeline. It provides real-time visibility into the current order being prepared and delivered.



View -- Order History Detail List (file: HFM/views/customer-order-history.ejs)

The history fragment renders all non-pending (completed/historical) orders in reverse chronological order. Each order card displays order ID, restaurant name, order date, status, and totals. Cards are expandable to show line-item details. This is the main historical review view for UC-C04.



Use Case 7 (UC-C05): Edit User Information

1. The Route

Files: HFM/routes/profile.js

Profile editing starts at GET /profile, which requires authentication and returns the current profile dataset for display/edit mode. Two update routes handle persistence: PUT /profile/personal for identity/contact fields and PUT /profile/address for default address fields. Each update route sanitizes input, applies field-level validation rules, writes changes to the user record, and returns a normalized JSON payload so the front-end can refresh profile state without reloading the full page.

2. The Controller

File: HFM/routes/profile.js | Function: router.get('/profile'), router.put('/profile/personal'), router.put('/profile/address')

The controller is responsible for all business logic. It executes the following steps:

1. Validates active session with requireLogin and loads current user profile by session userId.
2. Transforms user entity to a stable profile DTO for view/API output.

3. Personal-update handler sanitizes first/last name, phone, and date fields with cleanString().
4. Enforces required firstName and lastName before committing personal updates.
5. Address-update handler sanitizes address fields, normalizes state to uppercase, and persists updates.
6. Returns JSON success payloads with normalized profile data for client-side UI refresh.

3. The Models

This use case reads from and writes to the following model(s):

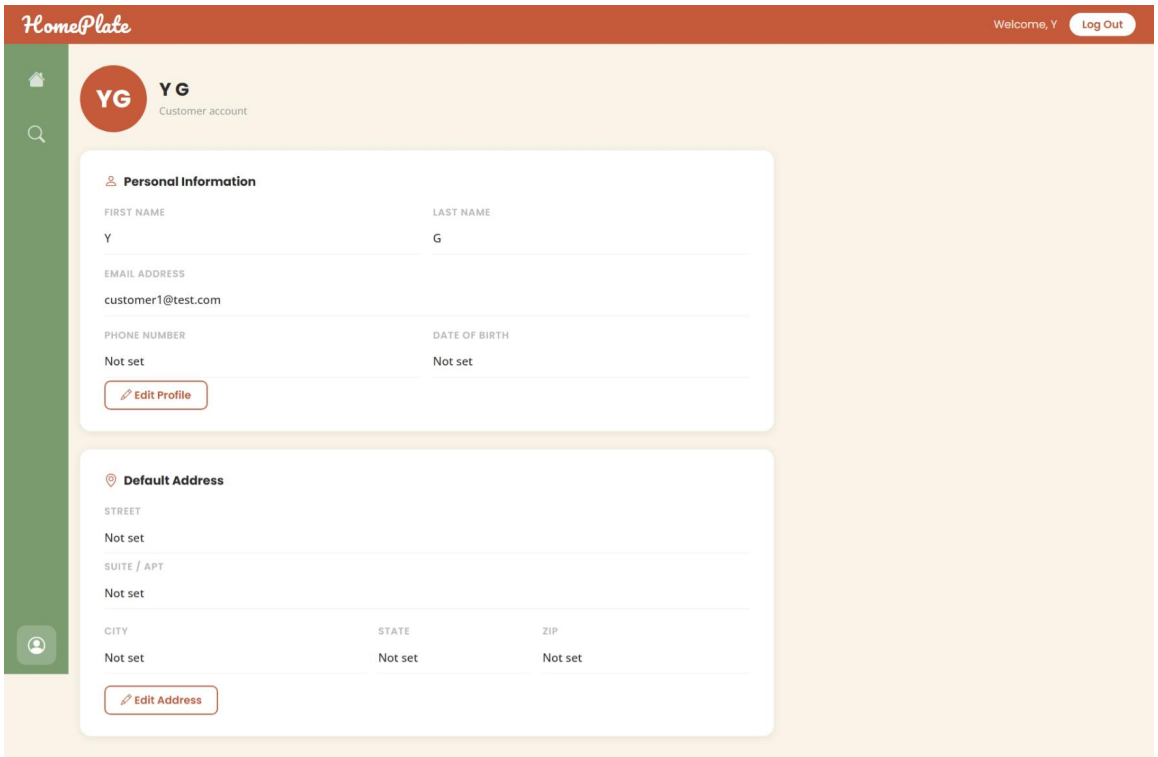
Model	Operation	What Happens
User	Read/Update	Reads current profile and persists personal/address edits.

4. The Views

This use case involves the following view(s):

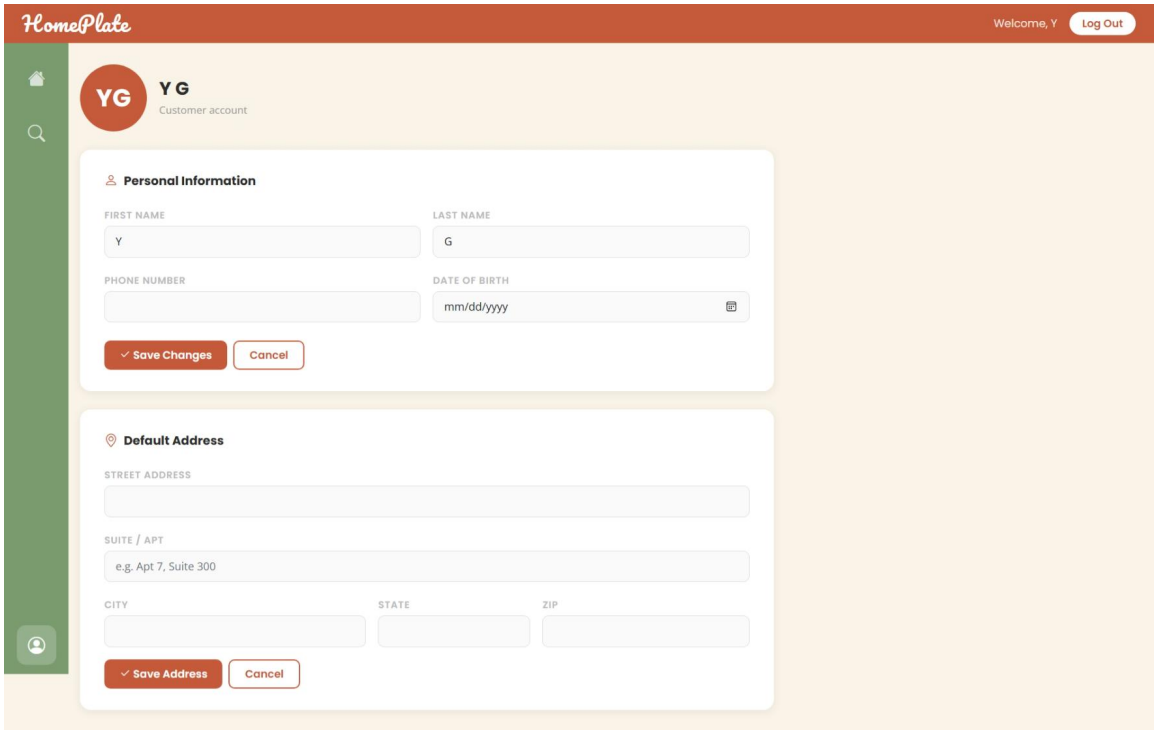
View -- Profile Information Display (file: HFM/views/profile.ejs)

This state displays current personal information (first name, last name, email, phone, date of birth) and default address in read-only cards. It is the starting point before entering edit mode in UC-C05. Each section has an "Edit" button to toggle into edit mode.



View -- Profile Edit Mode (file: HFM/views/profile.ejs)

When edit mode is enabled by clicking Edit, the same page reveals editable fields and save/cancel actions for both personal and address sections. Submitting updates calls profile PUT endpoints (/profile/personal or /profile/address) and refreshes displayed values after successful update.



Use Case 8 (UC-M01): Add Menu Item

1. The Route

Files: HFM/routes/menu.js, HFM/routes/merchant.js

Merchants reach menu management from dashboard routes and operate within a selected restaurant context stored in session. The creation action is handled by POST /api/menu, which is protected by cook-role authorization and restaurant selection checks. The route validates required item inputs, creates a restaurant-scoped menu record tied to the current merchant account, and returns JSON used by the UI to refresh the menu list and display the newly added item.

2. The Controller

File: HFM/routes/menu.js, HFM/routes/merchant.js | Function: router.post('/api/menu'), router.get('/merchant-my-menu')

The controller is responsible for all business logic. It executes the following steps:

1. Uses requireCook middleware to enforce cook role and selected restaurant context.
2. Reads name, price, quantity, category, and description from request payload.
3. Validates required name/price fields before attempting persistence.
4. Creates a new MenuItem with merchant userId and session restaurantId ownership binding.

5. Returns JSON for front-end refresh; merchant menu fragment endpoint reloads current restaurant items.

3. The Models

This use case reads from and writes to the following model(s):

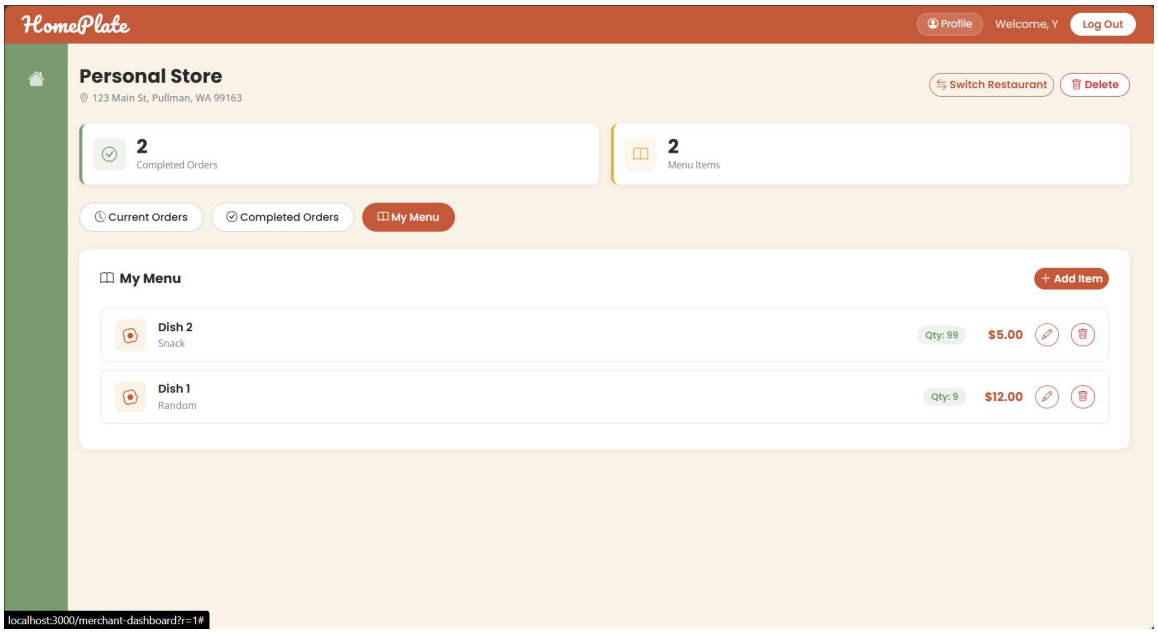
Model	Operation	What Happens
MenuItem	Create/Read	Creates new menu row and reloads menu list fragment.
Restaurant	Read	Confirms selected restaurant scope for item ownership.

4. The Views

This use case involves the following view(s):

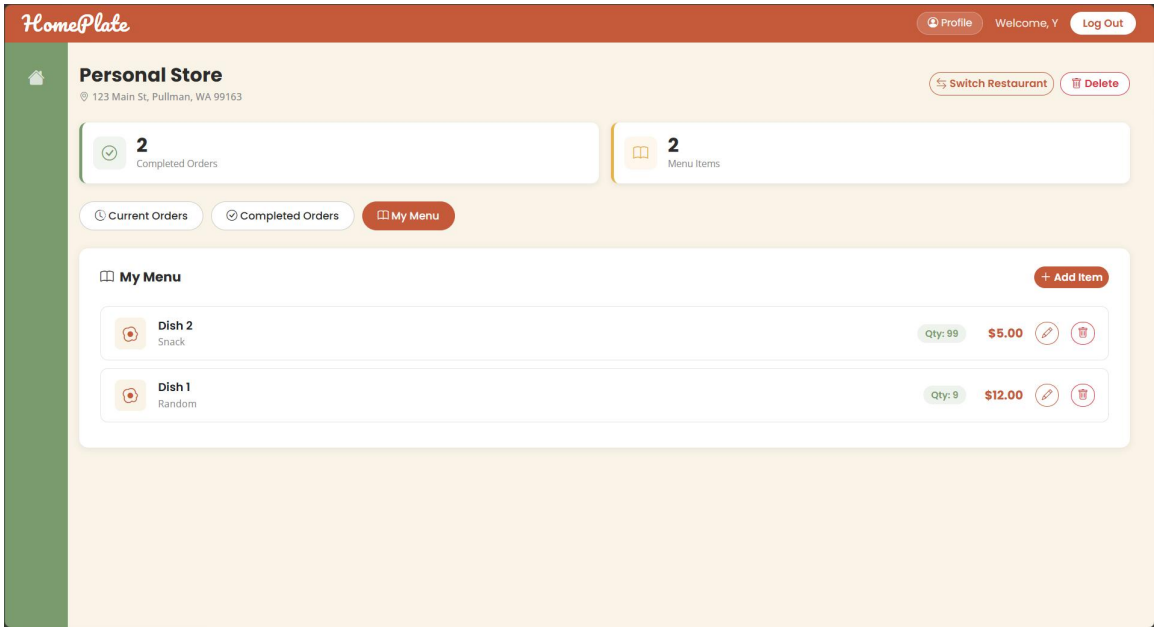
View -- Merchant Dashboard - Restaurant and Tab Selection (file: HFM/views/merchant-dashboard.ejs)

Merchant starts at the dashboard where they can select a restaurant to manage. Once a restaurant is selected, tabs appear for My Menu, Current Orders, Completed Orders, and other sections. The merchant clicks the "My Menu" tab to enter the menu management workspace. This is the entry point for UC-M01.



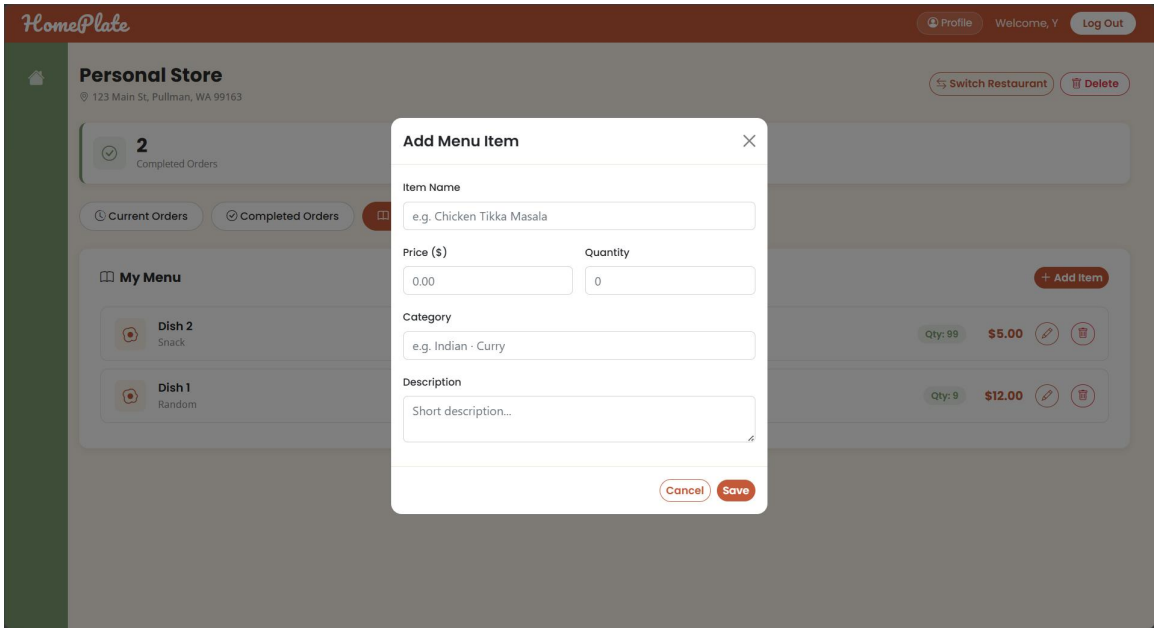
View -- My Menu Item List with Add Control (file: HFM/views/merchant-my-menu.ejs)

The My Menu view displays a list of existing menu items for the selected restaurant in a table or card format, showing name, price, quantity, category, and description. An "Add New Item" button or modal trigger is available. The view supports adding, editing, and deleting items.



View -- Add Menu Item Modal Form (file: HFM/views/merchant-my-menu.ejs)

The Add Item modal form captures name, price, quantity, category, and description fields for a new menu item. The form includes validation for required fields. Clicking Save triggers POST /api/menu and refreshes the menu list to display the newly created item.



Use Case 9 (UC-M02): Delete Menu Item

1. The Route

Files: HFM/routes/menu.js, HFM/views/merchant-my-menu.ejs

Deleting a menu item is triggered from the merchant menu UI and sent to DELETE /api/menu/:id. The route is protected by cook-role middleware and selected-restaurant context, then performs a scoped lookup using both item id and current restaurant id. If the item exists in scope, it is deleted and a success JSON response is returned; otherwise, the route returns a not-found response to prevent invalid or cross-scope deletion.

2. The Controller

File: HFM/routes/menu.js | Function: router.delete('/api/menu/:id')

The controller is responsible for all business logic. It executes the following steps:

1. Applies requireCook authorization and selected restaurant precondition.
2. Finds target MenuItem by both id and session restaurantId scope.
3. Returns 404 if item is not found in the selected restaurant scope.
4. Destroys the scoped menu item and returns success JSON for UI refresh.

3. The Models

This use case reads from and writes to the following model(s):

Model	Operation	What Happens
MenuItem	Read/Delete	Finds scoped item record

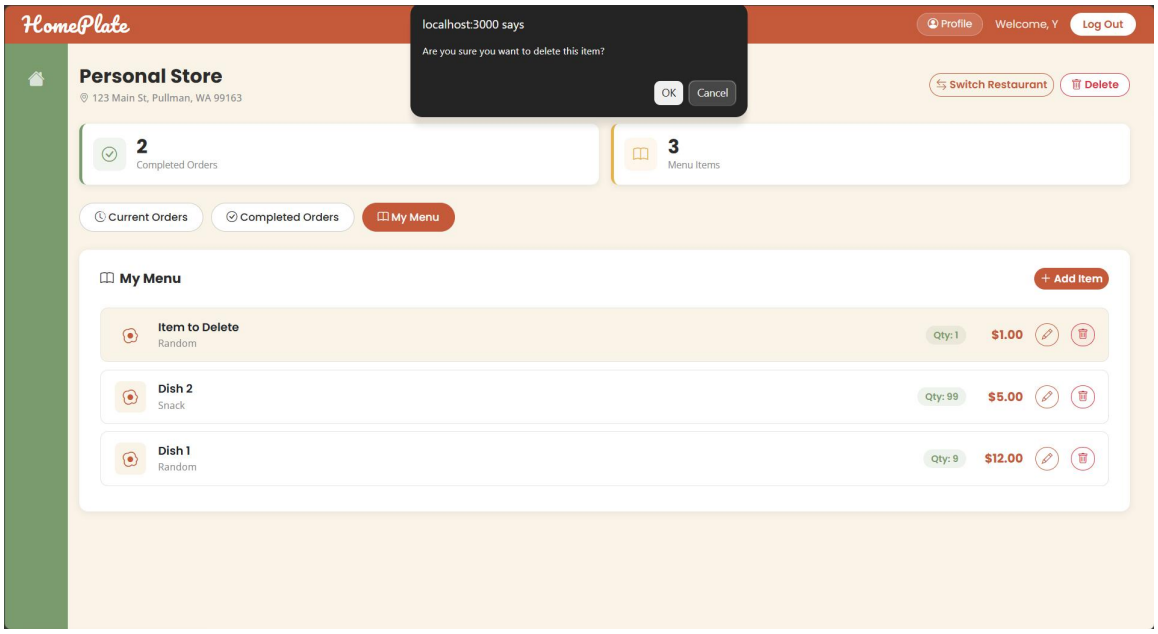
		and permanently removes it.
Restaurant	Read	Provides session scope constraint for safe deletion.

4. The Views

This use case involves the following view(s):

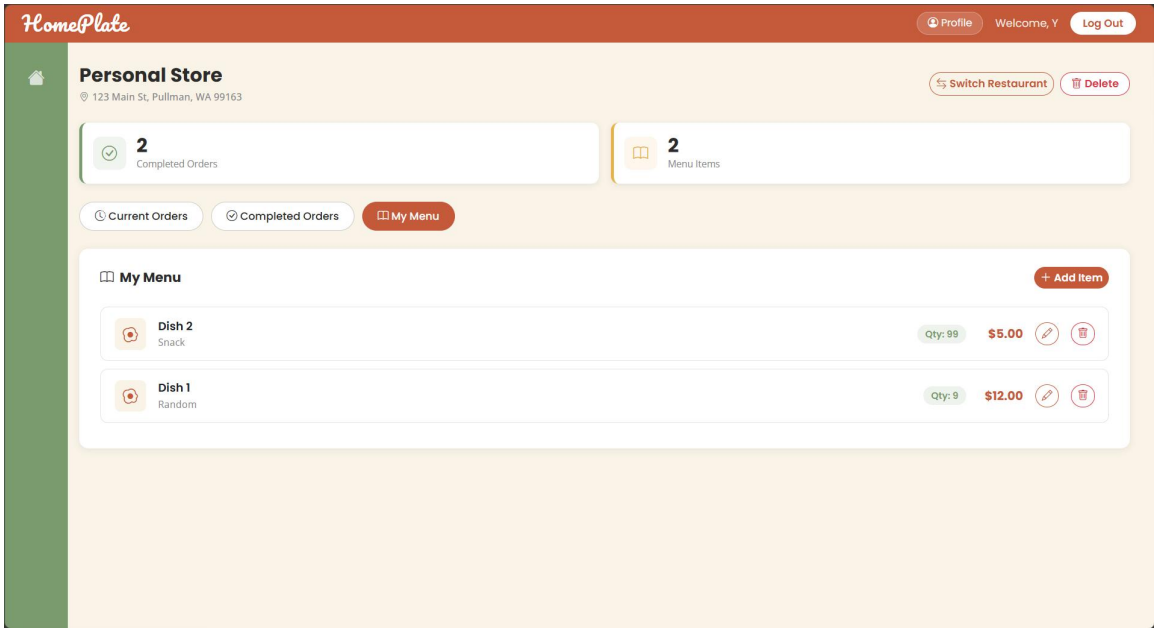
View -- My Menu List with Delete Trigger (file: HFM/views/merchant-my-menu.ejs)

The merchant sees existing menu items in a list/table view. Each item row displays name, price, quantity, category, and description. Each row includes a delete button or icon (trash icon, "Delete" link). Clicking the delete button triggers the DELETE /api/menu/:id action for UC-M02.



View -- My Menu List After Deletion (file: HFM/views/merchant-my-menu.ejs)

After successful DELETE /api/menu/:id, the menu fragment refreshes automatically via JavaScript. The deleted item no longer appears in the list, and the menu count is recalculated and updated. This confirms delete completion.



Use Case 10 (UC-M03): Edit Item

1. The Route

Files: HFM/routes/menu.js, HFM/views/merchant-my-menu.ejs

Item editing is submitted to PUT /api/menu/:id from the merchant menu interface. The route first enforces cook authorization and current restaurant selection, then loads the target row within that restaurant scope. Incoming editable fields (name, price, quantity, category, description) are conditionally applied, persisted, and returned as JSON so the front-end can re-render the updated item values in place.

2. The Controller

File: HFM/routes/menu.js | Function: router.put('/api/menu/:id')

The controller is responsible for all business logic. It executes the following steps:

1. Applies requireCook authorization and selected restaurant precondition.
2. Loads the menu record by id + restaurant scope to prevent cross-restaurant updates.
3. Maps optional payload fields (name, price, quantity, category, description) to model properties.
4. Persists the updated item and returns the updated JSON object to the front-end.

3. The Models

This use case reads from and writes to the following model(s):

Model	Operation	What Happens

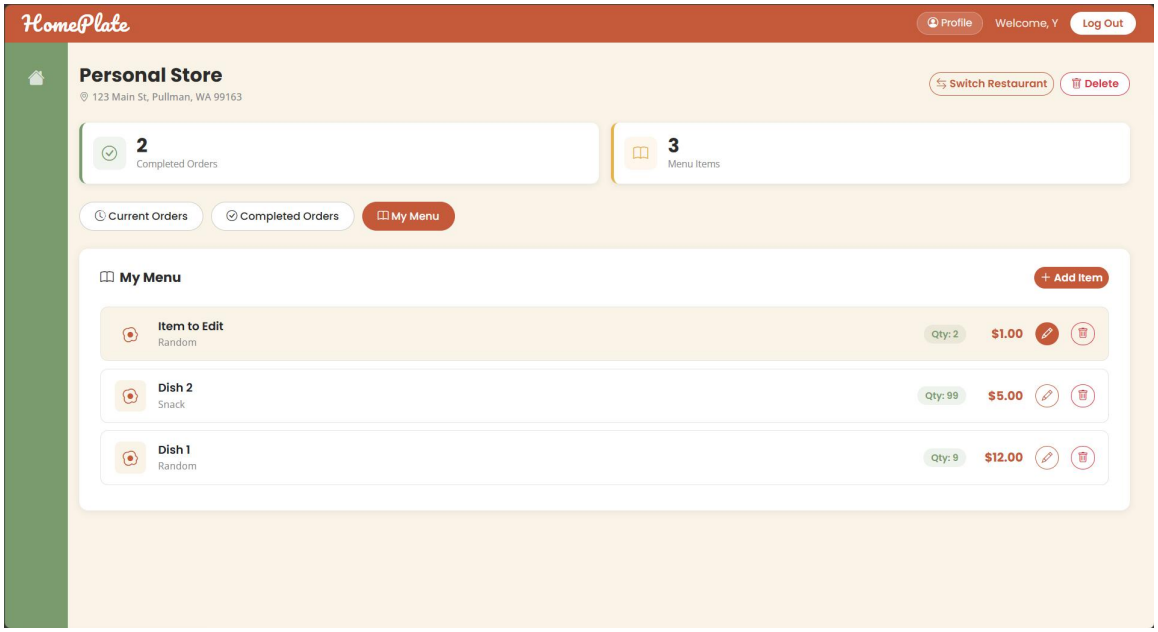
MenuItem	Read/Update	Loads selected item and writes edited fields.
Restaurant	Read	Enforces same-restaurant ownership during updates.

4. The Views

This use case involves the following view(s):

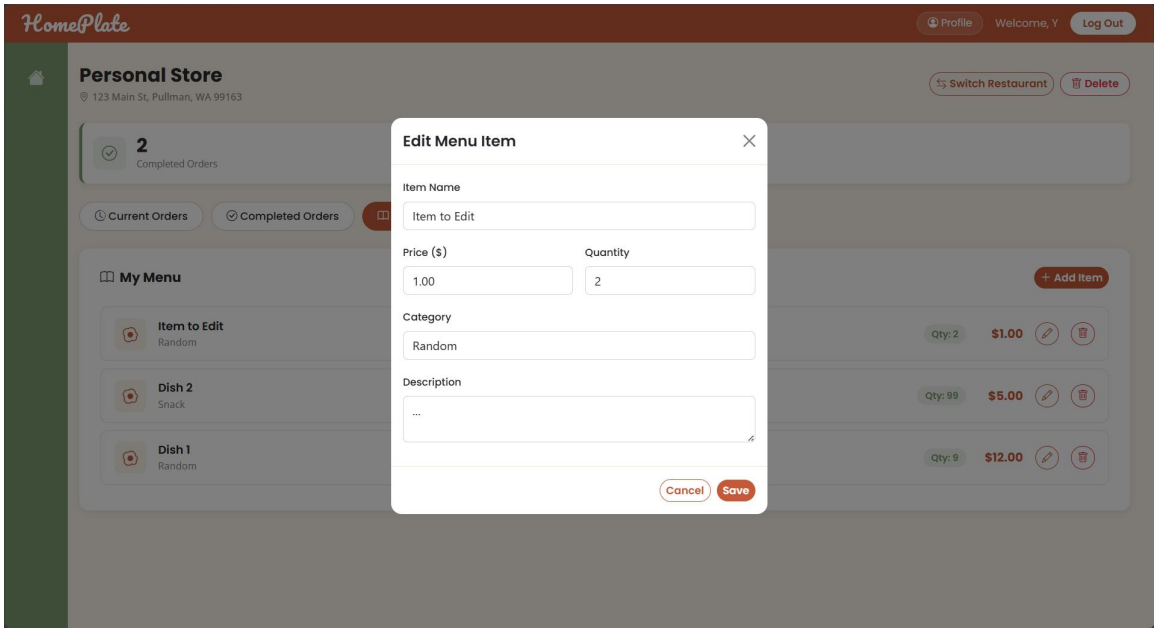
View -- My Menu List with Edit Trigger (file: HFM/views/merchant-my-menu.ejs)

The merchant views the menu item list where each row shows name, price, quantity, category, and description. Each row includes an edit button or icon. Clicking edit opens the edit modal prefilled with the item's current values.



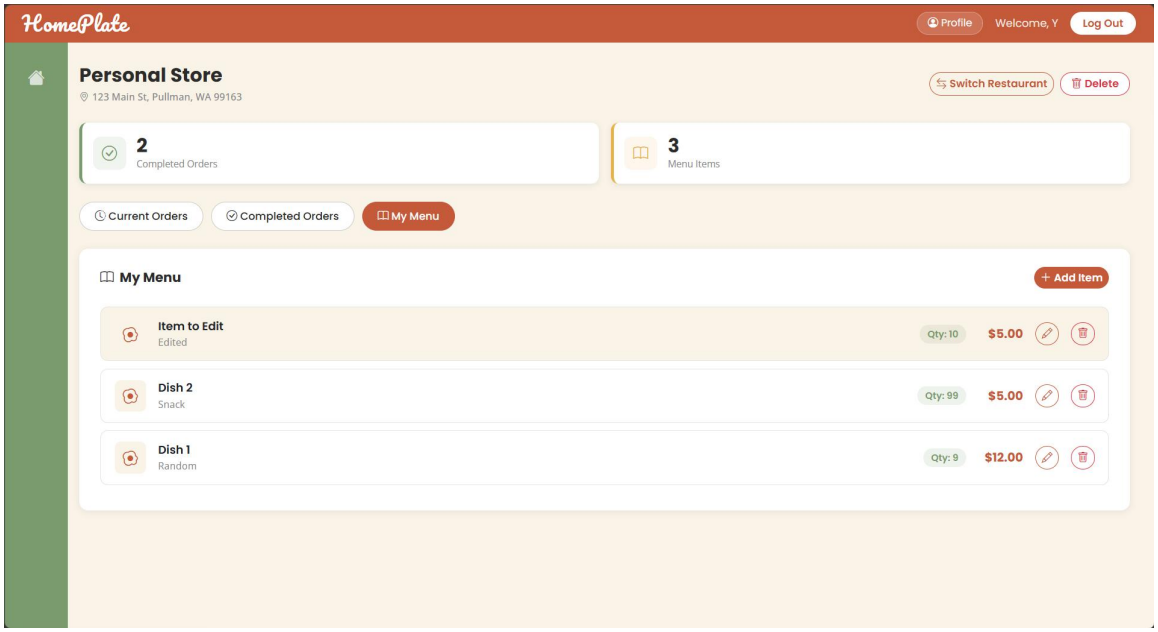
View -- Edit Menu Item Modal Form (Prefilled) (file: HFM/views/merchant-my-menu.ejs)

The edit modal opens with all fields (name, price, quantity, category, description) prefilled with the item's current values. The merchant can modify any field. Clicking Save triggers PUT /api/menu/:id and refreshes the menu list with updated values.



View -- My Menu List After Update (file: HFM/views/merchant-my-menu.ejs)

After PUT /api/menu/:id succeeds, the menu fragment refreshes automatically via JavaScript. The updated item values (name, price, category, quantity) appear in the refreshed list, confirming the update completion.



Use Case 11 (UC-M04): View Reports

1. The Route

Files: HFM/views/merchant-reports.html, HFM/routes/merchant.js

Reporting data is exposed through GET /merchant-completed-orders for the selected merchant restaurant. After authorization and restaurant-context checks, the route loads delivered orders in reverse chronological order and enriches each order with its line items. The resulting dataset is rendered as the completed-orders report view used by merchants to inspect historical performance, while advanced aggregated report APIs remain outside the current route implementation.

2. The Controller

File: HFM/routes/merchant.js | Function: router.get('/merchant-completed-orders')

The controller is responsible for all business logic. It executes the following steps:

1. Applies requireCook middleware and validates selected restaurant from session context.
2. Queries delivered orders for the selected restaurant in descending createdAt order.
3. Loads line items for each completed order and merges detail payload for reporting display.
4. Renders merchant-completed-orders fragment as the implemented reporting data view.

3. The Models

This use case reads from and writes to the following model(s):

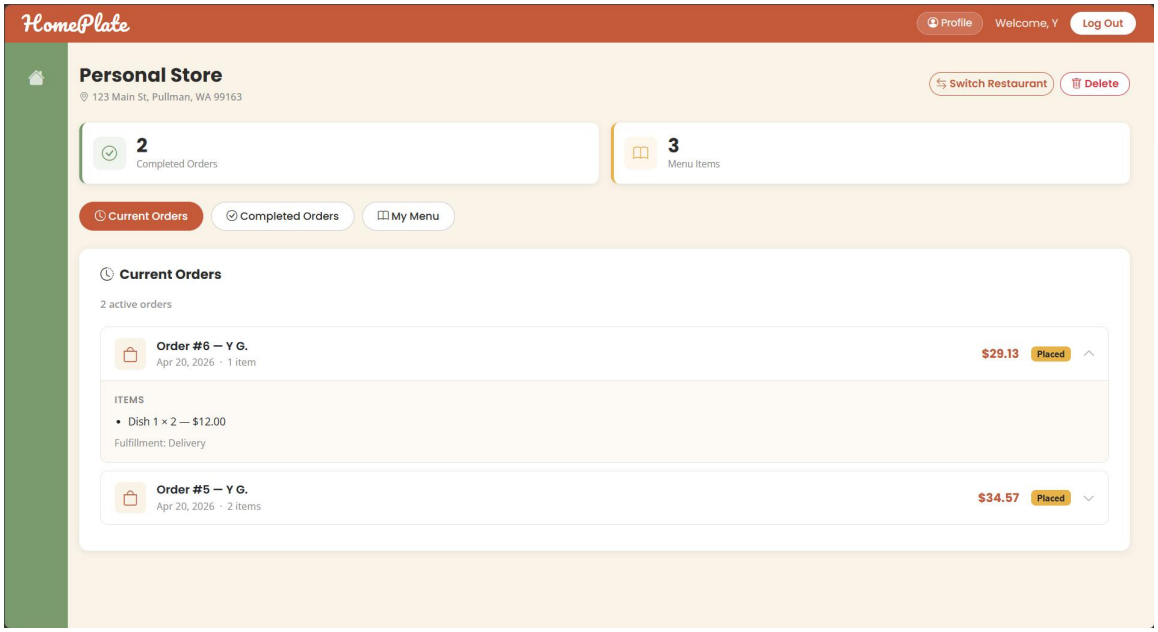
Model	Operation	What Happens
Order	Read	Fetches delivered/completed orders for report-oriented review.
OrderItem	Read	Provides item-level details used in report displays.
Restaurant	Read (Scope)	Ensures report data is restaurant-specific.

4. The Views

This use case involves the following view(s):

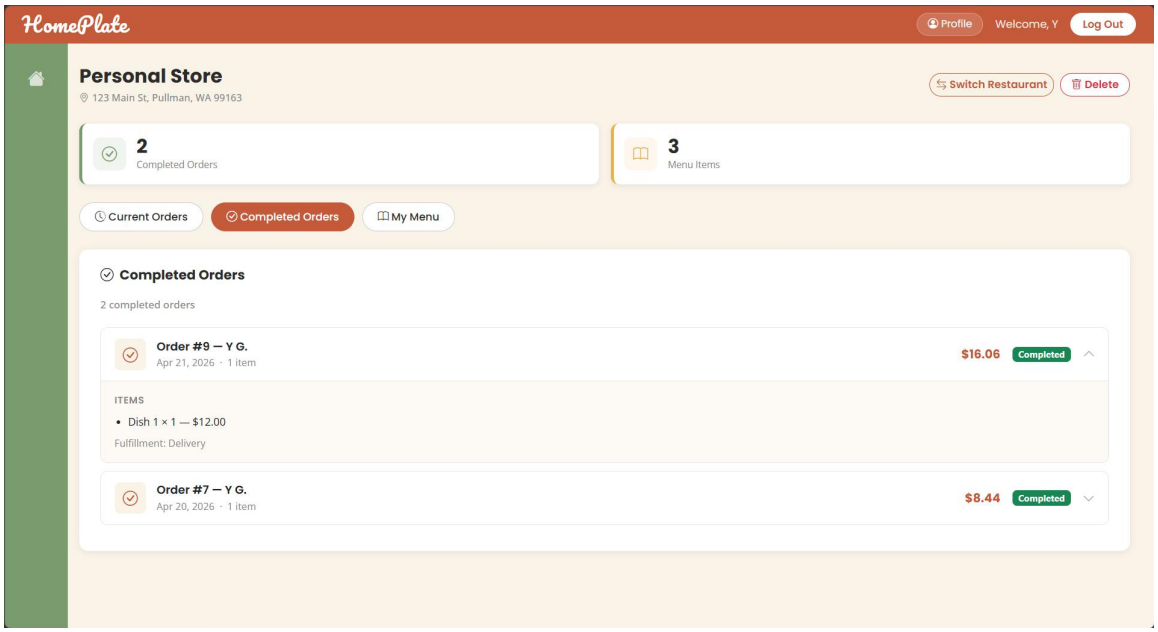
View -- Merchant Dashboard - Current Orders (file: merchant-dashboard.ejs)

From the merchant dashboard, the merchant can click the Current Orders tab to open the active order queue. This dashboard view acts as the operational entry point for order fulfillment, where the merchant can switch between Current Orders, Completed Orders, and My Menu while staying in the selected restaurant context.



View -- Merchant Dashboard - Completed Orders (file: HFM/views/merchant-dashboard.ejs)

From the merchant dashboard, the merchant can navigate to view completed orders or access reporting features. This implemented view provides delivered-order history and line-item details, which serves as the current reporting data source for UC-M04. Each completed order card displays order ID, completion date, restaurant name, line items with quantities and prices, and total amounts.



Use Case 12 (UC-M05): Edit Restaurant Information

1. The Route

Files: HFM/routes/merchant.js, HFM/views/merchant-dashboard.ejs

Restaurant management is routed through dashboard and restaurant-management endpoints in merchant routes. GET /merchant-dashboard loads owned restaurants and selected-restaurant state, POST /merchant-restaurant creates a new restaurant with normalized tags, and POST /merchant-restaurant/delete removes a selected restaurant after ownership validation. The delete path performs transactional cleanup of related records so the UI can safely return to restaurant selection state.

2. The Controller

File: HFM/routes/merchant.js | Function: router.get('/merchant-dashboard'), router.post('/merchant-restaurant'), router.post('/merchant-restaurant/delete')

The controller is responsible for all business logic. It executes the following steps:

1. Dashboard handler loads all restaurants owned by the logged-in cook and manages selected restaurant state in session.
2. Create-restaurant handler validates required fields, normalizes tag list, creates restaurant, and selects it in session.
3. Delete-restaurant handler verifies ownership of target restaurant before deletion.
4. Executes transactional cascade cleanup: deletes related OrderItem rows, then Orders, then MenuItems, then Restaurant.
5. Clears session restaurant selection when deleted restaurant was currently selected, then redirects to restaurant picker state.

3. The Models

This use case reads from and writes to the following model(s):

Model	Operation	What Happens
Restaurant	Create/Read/Delete	Creates stores, lists owned stores, and deletes selected store.
MenuItem	Delete (Cascade)	Removes menu rows when a restaurant is deleted.
Order	Delete (Cascade)	Removes restaurant orders during store deletion flow.
OrderItem	Delete (Cascade)	Removes order-item rows associated with deleted

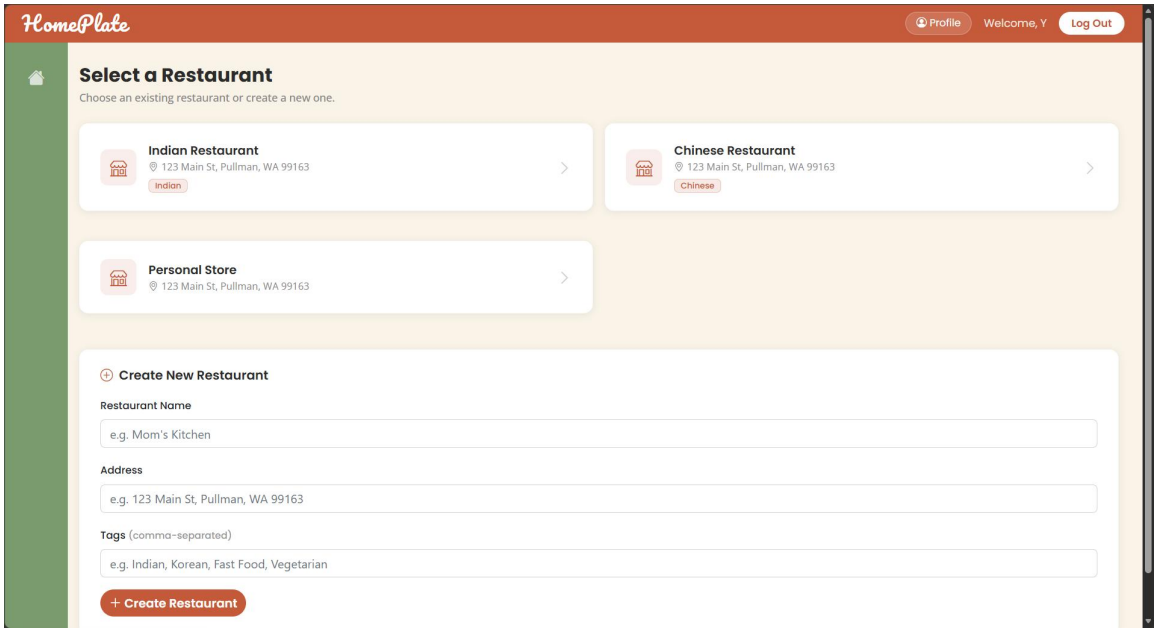
store orders.

4. The Views

This use case involves the following view(s):

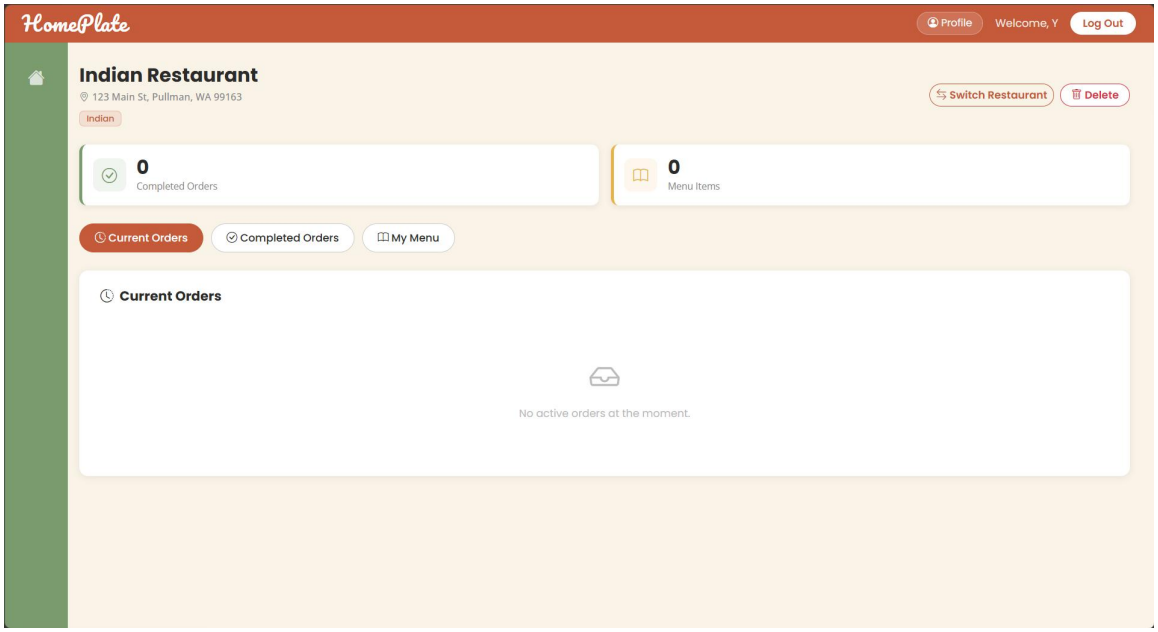
View -- Restaurant Picker / Setup View (file: HFM/views/merchant-dashboard.ejs)

This state lets the merchant select an existing restaurant, create a new one with name, address, and tags, or delete a restaurant. The view displays all owned restaurants as selectable cards and includes a form for creating a new restaurant. This is the implemented entry behavior related to UC-M05.



View -- Selected Restaurant Header and Controls (file: HFM/views/merchant-dashboard.ejs)

After selecting a restaurant, the dashboard header renders the active restaurant information (name, address, tags). Management controls are displayed including Switch Restaurant button and Delete Restaurant button. This view reflects current read/manage behavior for restaurant info.



2.2 Section 2: Route / API Table

Routes are grouped by feature area for readability. Each table includes method, path, behavior, and authentication/role requirement.

Core / Utility

Method	Route	Description	Auth Required
GET	/	Render the landing page.	No
GET	/users	Return default users route response.	No

Authentication

Method	Route	Description	Auth Required
GET	/login	Render the login page.	No
POST	/login	Authenticate user and create session.	No
GET	/register	Render the registration page.	No

POST	/register	Create a new customer or cook account.	No
GET	/logout	Destroy current session and redirect to login.	Yes -- Logged-in User

Restaurant Discovery

Method	Route	Description	Auth Required
GET	/browse	Browse/search restaurants and optional tag filter.	No
GET	/restaurant-detail	Render restaurant detail page with available menu items.	No

Cart and Checkout (Customer)

Method	Route	Description	Auth Required
GET	/cart	Render current pending cart groups for session user.	Yes -- Logged-in User (Customer Flow)
POST	/cart/add	Add menu item to cart (create/increment pending order item).	Yes -- Logged-in User (Customer Flow)
PUT	/cart/item/:id	Update cart item quantity and recalculate totals.	Yes -- Logged-in User (Customer Flow)
DELETE	/cart/item/:id	Remove item from cart and delete empty pending order.	Yes -- Logged-in User (Customer Flow)

GET	/checkout	Render checkout page for selected/latest pending order.	Yes -- Logged-in User (Customer Flow)
POST	/checkout	Validate delivery/order, place order, and deduct inventory.	Yes -- Logged-in User (Customer Flow)
GET	/order-confirmation	Render confirmation for a placed order.	Yes -- Logged-in User (Customer Flow)

Customer Dashboard and Order History

Method	Route	Description	Auth Required
GET	/customer-dashboard	Render customer dashboard with active/history counters.	Yes -- Customer
GET	/customer-current-order	Render latest non-pending order status fragment.	Yes -- Customer
GET	/customer-order-history-data	Render customer order-history fragment.	Yes -- Customer

Profile

Method	Route	Description	Auth Required
GET	/profile	Render user profile page.	Yes -- Logged-in User
PUT	/profile/personal	Update personal profile fields.	Yes -- Logged-in User
PUT	/profile/address	Update address profile fields.	Yes -- Logged-in User

Merchant Dashboard

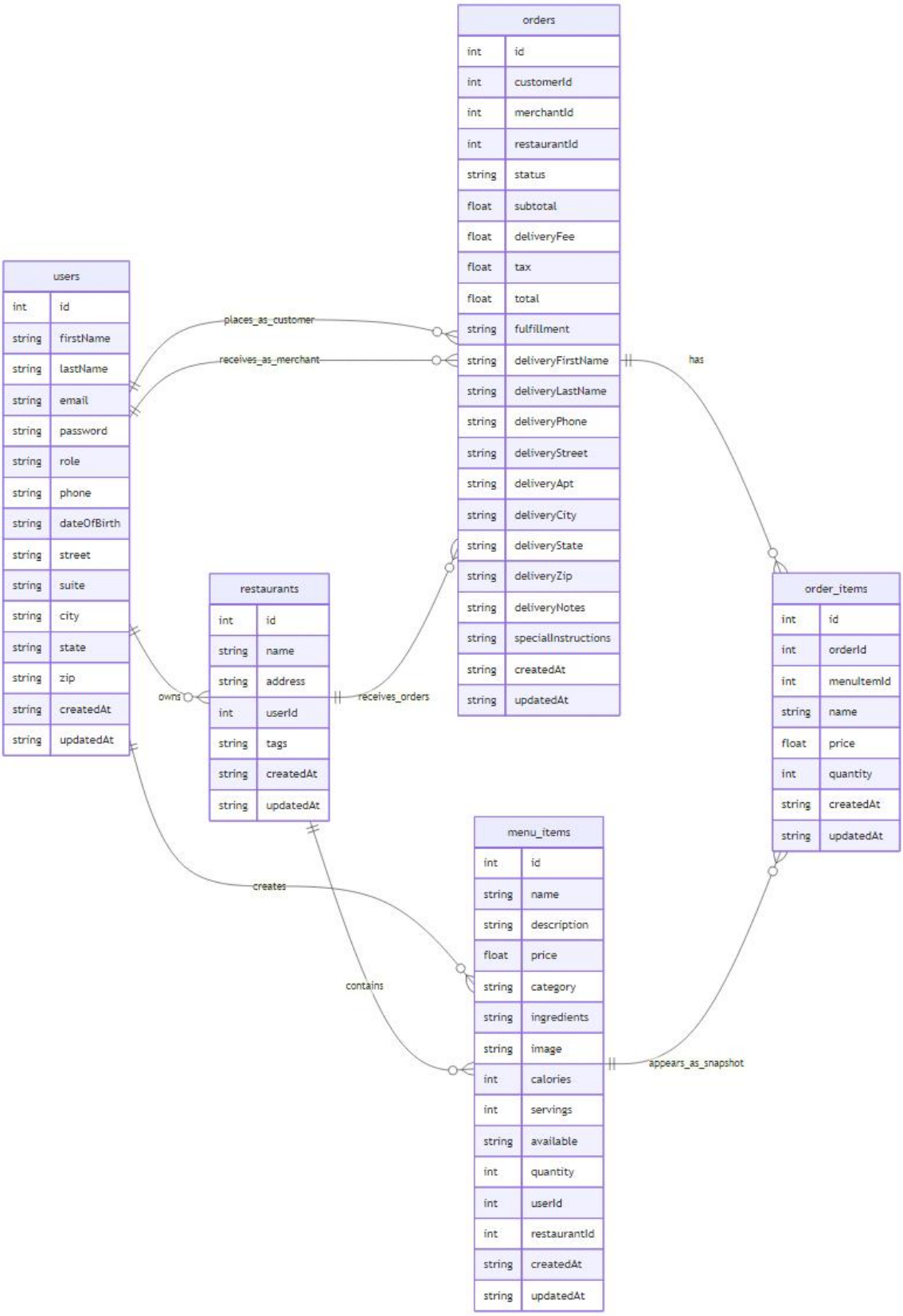
Method	Route	Description	Auth Required
GET	/merchant-dashboard	Render Merchant dashboard and restaurant selector.	Yes -- Merchant (Cook)
POST	/merchant-restaurant	Create a restaurant for the logged-in Merchant.	Yes -- Merchant (Cook)
POST	/merchant-restaurant/delete	Delete a Merchant restaurant and cascade related data.	Yes -- Merchant (Cook)
GET	/merchant-my-menu	Render Merchant menu fragment for selected restaurant.	Yes -- Merchant (Cook)
GET	/merchant-current-orders	Render active current orders fragment for Merchant.	Yes -- Merchant (Cook)
GET	/merchant-completed-orders	Render completed orders fragment for Merchant.	Yes -- Merchant (Cook)

Merchant Menu API

Method	Route	Description	Auth Required
GET	/api/menu	List menu items for selected restaurant.	Yes -- Merchant (Cook)
POST	/api/menu	Create menu item for selected restaurant.	Yes -- Merchant (Cook)
PUT	/api/menu/:id	Update a scoped menu item.	Yes -- Merchant (Cook)
DELETE	/api/menu/:id	Delete a scoped menu item.	Yes -- Merchant (Cook)

2.3 Section 3: Data Model / ER Diagram

This diagram uses the actual database tables in the application. It shows the main tables and their relationships: users, restaurants, menu_items, orders, and order_items.


orders
int id
int customerId
int merchantId
int restaurantId
string status
float subtotal
float deliveryFee
float tax
float total
string fulfillment
string deliveryFirstName
string deliveryLastName
string deliveryPhone
string deliveryStreet
string deliveryApt
string deliveryCity
string deliveryState
string deliveryZip
string deliveryNotes
string specialInstructions
string createdAt
string updatedAt
users
int id
string firstName
string lastName
string email
string password
string role
string phone
string dateOfBirth
string street
string suite
string city
string state
string zip
string createdAt
string updatedAt
restaurants
int id
string name
string address
int userId
string tags
string createdAt
string updatedAt
order_items
int id
int orderId
int menuItemId
string name
float price
int quantity
string createdAt
string updatedAt
menu_items
int id
string name
string description
float price
string category
string ingredients
string image
int calories
int servings
string available
int quantity
int userId
int restaurantId
string createdAt
string updatedAt
places_as_customer
receives_as_merchant
has
owns
receives_orders
creates
contains
appears_as_snapshot

One important detail is that the users table is shared by multiple Sequelize models in the code. It maps to User, Customer, and Merchant. Customer and Merchant are not separate tables. They are different role-based models built on the same users table.

The remaining tables map directly to their corresponding models: restaurants to Restaurant, menu_items to MenuItem, orders to Order, and order_items to OrderItem.

Model Descriptions:

User

Field	Type	Notes
id	INTEGER	Primary key, auto-increment
firstName	STRING(50)	Required
lastName	STRING(50)	Required
email	STRING(255)	Required, unique, login identifier
password	STRING(255)	Required, bcrypt hash
role	ENUM('customer','cook')	Required, default 'customer'
phone	STRING(20)	Optional
dateOfBirth	DATEONLY	Optional
street	STRING(255)	Optional
suite	STRING(100)	Optional
city	STRING(100)	Optional
state	STRING(2)	Optional, 2-letter code
zip	STRING(10)	Optional
createdAt	DATE	Auto-managed by Sequelize timestamp
updatedAt	DATE	Auto-managed by Sequelize timestamp

User is the base identity model and stores shared account/profile fields for all roles. Authentication reads this table by email and compares bcrypt-hashed passwords.

Customer

Field	Type	Notes
id	INTEGER	Primary key, auto-increment (same physical row in users table)
firstName	STRING(50)	Required
lastName	STRING(50)	Required
email	STRING(255)	Required, unique
password	STRING(255)	Required, bcrypt hash
role	ENUM('customer','cook')	Scoped/defaulted to 'customer'
phone	STRING(20)	Optional
dateOfBirth	DATEONLY	Optional
street	STRING(255)	Optional
suite	STRING(100)	Optional
city	STRING(100)	Optional
state	STRING(2)	Optional
zip	STRING(10)	Optional
createdAt	DATE	Auto-managed by Sequelize timestamp
updatedAt	DATE	Auto-managed by Sequelize timestamp

Customer is a role-scoped subclass of User that maps to the same users table. Its default scope enforces role='customer', so it behaves as a filtered view over user accounts.

Merchant

Field	Type	Notes
id	INTEGER	Primary key, auto-increment (same physical row in users table)
firstName	STRING(50)	Required
lastName	STRING(50)	Required
email	STRING(255)	Required, unique
password	STRING(255)	Required, bcrypt hash
role	ENUM('customer','cook')	Scoped/defaulted to 'cook'
phone	STRING(20)	Optional
dateOfBirth	DATEONLY	Optional
kitchenName	STRING(100)	Optional Merchant profile field
cuisine	STRING(50)	Optional
bio	TEXT	Optional
deliveryFee	DECIMAL(5,2)	Optional, default 0.00
availableTimes	STRING(20)	Optional
minimumOrder	DECIMAL(5,2)	Optional, default 0.00
street	STRING(255)	Optional
suite	STRING(100)	Optional
city	STRING(100)	Optional
state	STRING(2)	Optional
zip	STRING(10)	Optional
createdAt	DATE	Auto-managed by Sequelize timestamp
updatedAt	DATE	Auto-managed by Sequelize timestamp

Merchant is a role-scoped subclass of User for cook/Merchant accounts, also stored in users. It adds kitchen/business metadata used by dashboard and listing flows.

Restaurant

Field	Type	Notes
id	INTEGER	Primary key, auto-increment
name	STRING(100)	Required restaurant name
address	STRING(255)	Required display address
userId	INTEGER	Owner user id (Merchant), logical FK -> users.id
tags	TEXT	Optional JSON array stored as string, default []
createdAt	DATE	Auto-managed by Sequelize timestamp
updatedAt	DATE	Auto-managed by Sequelize timestamp

Restaurant represents a Merchant-owned storefront. Tags are serialized JSON for flexible filtering in browse/search views.

MenuItem

Field	Type	Notes
id	INTEGER	Primary key, auto-increment
name	STRING(100)	Required item name
description	TEXT	Optional
price	DECIMAL(10,2)	Required unit price
category	STRING(50)	Optional

ingredients	TEXT	Optional
image	STRING(255)	Optional image path/URL
calories	INTEGER	Optional
servings	INTEGER	Optional, default 1
available	BOOLEAN	Required, default true
quantity	INTEGER	Required stock count, default 0
userId	INTEGER	Merchant owner id, logical FK -> users.id
restaurantId	INTEGER	Restaurant id, logical FK -> restaurants.id
createdAt	DATE	Auto-managed by Sequelize timestamp
updatedAt	DATE	Auto-managed by Sequelize timestamp

MenuItem stores sellable products under a restaurant and merchant owner. Inventory and availability are updated during cart and checkout flows.

Order

Field	Type	Notes
id	INTEGER	Primary key, auto-increment
customerId	INTEGER	Customer id, logical FK -> users.id
merchantId	INTEGER	Merchant id, logical FK -> users.id
restaurantId	INTEGER	Optional logical FK -> restaurants.id
status	ENUM('pending','placed','confirmed','ready','delivered','	Required, default 'pending'

	cancelled')	
subtotal	DECIMAL(10,2)	Required, default 0.00
deliveryFee	DECIMAL(5,2)	Required, default 0.00
tax	DECIMAL(10,2)	Required, default 0.00
total	DECIMAL(10,2)	Required, default 0.00
fulfillment	ENUM('delivery','pickup')	Required, default 'delivery'
deliveryFirstName	STRING(50)	Optional snapshot field
deliveryLastName	STRING(50)	Optional snapshot field
deliveryPhone	STRING(20)	Optional snapshot field
deliveryStreet	STRING(255)	Optional snapshot field
deliveryApt	STRING(100)	Optional snapshot field
deliveryCity	STRING(100)	Optional snapshot field
deliveryState	STRING(2)	Optional snapshot field
deliveryZip	STRING(10)	Optional snapshot field
deliveryNotes	TEXT	Optional
specialInstructions	TEXT	Optional
createdAt	DATE	Auto-managed by Sequelize timestamp
updatedAt	DATE	Auto-managed by Sequelize timestamp

Order is both the cart container (status='pending') and the finalized purchase record. It stores monetary totals and delivery snapshot fields to preserve historical accuracy.

OrderItem

Field	Type	Notes
id	INTEGER	Primary key, auto-increment

orderId	INTEGER	Logical FK -> orders.id
menuItemId	INTEGER	Logical FK -> menu_items.id
name	STRING(100)	Item name snapshot at purchase time
price	DECIMAL(10,2)	Item price snapshot at purchase time
quantity	INTEGER	Required quantity, default 1
createdAt	DATE	Auto-managed by Sequelize timestamp
updatedAt	DATE	Auto-managed by Sequelize timestamp

OrderItem is the line-item table for each order. Name and price are denormalized snapshots so historical orders remain accurate when menu metadata changes later.

Relationship Summary

Relationship	Type	Description
User -> Restaurant	One-to-Many	One Merchant user can own many restaurants via restaurants.userId.
User -> MenuItem	One-to-Many	One Merchant user can create many menu items via menu_items.userId.
Restaurant -> MenuItem	One-to-Many	One restaurant can have many menu items via menu_items.restaurantId.
User(customer) -> Order	One-to-Many	One customer user can place many orders via orders.customerId.
User(Merchant) -> Order	One-to-Many	One Merchant user can receive many orders via orders.merchantId.

Restaurant -> Order	One-to-Many	One restaurant can have many related orders via orders.restaurantId.
Order -> OrderItem	One-to-Many	One order contains many line items via order_items.orderId.
MenuItem -> OrderItem	One-to-Many	One menu item may appear in many order items via order_items.menuItemId.
User -> Customer	Specialization (scope)	Customer model is a role-filtered view of users (role='customer').
User -> Merchant	Specialization (scope)	Merchant model is a role-filtered view of users (role='cook').

Design note: the current codebase uses logical foreign-key relationships in model fields and route logic, but does not define explicit Sequelize association methods (hasMany/belongsTo).

2.4 Section 4: Known Issues and Limitations

This section lists incomplete features and a focused set of unhandled edge cases discovered during code review.

Incomplete Features

- Admin module (all admin-side features) was removed with instructor approval to keep scope aligned with project priorities.
- Delivery driver module (all driver-side features) was removed with instructor approval.
- Profile: Add payment method was not implemented because it is out of scope for the current submission.
- Profile: Delete account was not implemented because it is out of scope and not a hard requirement.
- Profile: Making profile page cusotmer-only or role-specific was not implemented because of time constraints.
- Password recovery flow was not implemented because it is out of scope.
- Customer dashboard: order-status progress tracker was removed with instructor approval.

- Merchant order-state actions (confirm/reject/update status) were removed with instructor approval.
- Merchant storefront open/close status toggle was removed with instructor approval.
- Browse page restaurant sorting was removed with instructor approval.
- Search results filtering by restaurant category attributes (rating, price, distance) was not completed due to time constraints.
- Customer reviews authoring feature was removed with instructor approval.

Bugs

- POST /cart/add does not verify that menuItemId belongs to the submitted restaurantId, so a mismatched item/restaurant combination can be inserted into the same order. Updates to fix this were not implemented due to time constraints.
- Menu create/update endpoints parse numeric fields without strict validation; invalid numeric input can propagate to database write errors and return generic 500 responses instead of clear 4xx validation errors. Updates to fix this were not implemented due to time constraints.

Unhandled Edge Cases

- Checkout state normalization falls back to the first two characters for unknown input (for example, 'InvalidState' becomes 'IN'), which can silently store incorrect addresses. Logics to handle this were not implemented due to time constraints
- Cart quantity update does not validate that quantity is a valid positive integer before assignment, so malformed input can trigger server-side failures instead of a clean 4xx validation response. Rules to handle this were not implemented due to time constraints.
- Customer order-history endpoint loads all non-pending orders with per-order follow-up queries (items and restaurant), which creates an N+1 pattern and can degrade heavily for large histories. Rules to handle this were not implemented due to time constraints.
- Browse page rendering performs per-restaurant count and owner lookups in a loop, creating query amplification that can become slow under larger restaurant datasets. Rules to handle this were not implemented due to time constraints.
- Logout does not handle session-destroy callback errors explicitly, so session termination failures are not surfaced to the user. Rules to handle this were not implemented due to time constraints.

Known Limitations

- Some pricing rules are hardcoded in server logic (for example tax rate and delivery fee in cart recalculation). Refactorings to resolve this were not implemented due to time constraints.
- Restaurant tags are stored as serialized JSON text and tag filtering is applied in application logic, which limits database-level indexing and advanced query flexibility for discovery features. Refactorings to resolve this were not implemented due to time constraints.

- Checkout is scoped to one pending order (effectively one restaurant order at a time), so a single unified checkout across multiple restaurants is not supported. Refactorings to resolve this were not implemented due to time constraints.